

Smish Guard

An Intelligent SMS Phishing Detection System



Neta Elbaz • Itamar Hadad

Project Advisor: Victor Taubkin

2026

Table of Contents

1. Introduction.....	5
1.1 Problem	5
1.2 Project Goals	5
1.3 Target Users.....	6
2. Literature Review and Research	7
2.1 Literature Review	7
2.2 Market Research.....	8
3. Proposed Solution	9
3.1 System Requirements.....	9
3.1.1 Functional Requirements	9
3.1.2 Non-Functional Requirements.....	9
3.2 Solution Overview.....	9
3.3 Solution Architecture	10
3.4 System Data Flow	11
4. Dataset.....	12
4.1 Dataset Sources	12
4.2 Dataset Structure	14
4.3 Dataset Distribution.....	15
4.4 Data Cleaning & Preprocessing	18
5. Feature Engineering	21
5.1 TF-IDF Features	22
5.2 SBERT Embeddings.....	23
5.3 URL Features	24
5.4 Phone Number Features	25
5.5 Email Features.....	25
5.6 Suspicious Keywords Features.....	26
5.7 Urgency and Threat Features.....	27
5.8 Message Structure Features.....	28
5.9 Additional Custom Features	29
5.10 Feature Importance Analysis.....	30
6. Machine Learning Model Development.....	33

6.1	Defining the Classification Problem	33
6.2	Text Preprocessing	33
6.3	Text Representation	33
6.3.1	TF-IDF Representation	34
6.3.2	SBERT Representation	34
6.4	Used Models	35
6.5	Model Training Process	36
6.6	Hyperparameter Tuning	37
6.7	Weighted Soft-Voting Ensemble	38
7.	Model Evaluation	40
7.1	Evaluation Methodology	40
7.2	Evaluation Metrics	40
7.2.1	Accuracy	41
7.2.2	Precision	41
7.2.3	Recall.....	42
7.2.4	F1-Score	42
7.2.5	Confusion Matrix	42
7.3	Results of the Selected Model.....	43
7.4	Comparison Between Models	44
7.5	Results Analysis	45
7.6	Conclusions from Model Evaluation.....	46
8.	Error Analysis.....	46
8.1	False Positives	46
8.2	False Negatives	47
8.3	Analysis of Failure Cases	48
8.4	Lessons Learned from Error Analysis.....	49
9.	Backend Server.....	49
9.1	Backend Overview	49
9.2	Technologies and Tools	50
9.3	Backend Architecture	50
9.4	Class Diagram UML	52
9.5	Main Components	52
9.5.1	Main Application	52

9.5.2	SMS Controller	52
9.5.3	SMS Analyzer Service	53
9.6	API Endpoints	53
10.	Android Application.....	54
10.1	Application Overview	54
10.2	Technologies and Tools	54
10.3	Application Architecture	54
10.4	Main Components	54
10.4.1	UI Layer	54
10.4.2	Business Layer	55
10.4.3	Data Layer	55
10.5	Permissions Management.....	56
10.6	User Interaction Flow.....	57
10.7	User Interface	58
11.	Development Challenges	60
11.1	Model and Dataset Challenges	60
11.2	Android Development Challenges	61
12.	Future Development.....	61
12.1	Support multiple SMS languages.....	61
12.2	Self-Improving Classification Model.....	61
13.	Conclusion	61
13.1	Planned vs. Actual Implementation	62
14.	Bibliography	63

1. Introduction

Our project focuses on trying to solve a common problem called smishing attacks.

Smishing attacks are fraud attempts using SMS, designed to steal sensitive information such as passwords, credit card details, and personal data.

This means that anyone who holds a mobile phone, 72% of global population, is vulnerable to this problem.

Fraud attempts can be carried out in various ways.

They may use phone calls, emails, face to face interactions, social media platforms, and more.

In this project, we decided to focus specifically on SMS based attacks, due to their widespread use and the increasing sophistication of malicious messages.

1.1 Problem

The primary problem addressed in this project is the growing difficulty of identifying smishing messages automatically.

This problem increases with the progress of AI in our world, enabling attackers to create more convincing and sophisticated phishing SMS messages, which are harder to detect.

Many malicious SMS messages are designed to appear legitimate and often bypass traditional spam filters. As a result, users may interact with harmful content and expose sensitive information.

After doing market research, we discovered that there is not a good enough solution today, that provides sufficiently accurate, automated, and real time detection of smishing attacks while remaining accessible to everyday users.

1.2 Project Goals

The main goal of this project is to provide a solution that is as automatic as possible and as accurate as possible for the passive detection of malicious SMS messages on user devices.

When suspicious message is identified, the system should send a live message notification to warn the user of the potential risk.

The solution should be easy to use, suitable for a wide audience, regardless of their technical knowledge.

In addition, the main part of this project is research, analyzing different machine learning approaches and comparing their performance.

1.3 Target Users

Our solution should fit anyone with android.

But there are some users that might benefit from it the most

1. Elderly users

Elderly users are often less familiar with digital threats, and modern technology changes very quickly, which makes it harder for them to keep up. This can make them more vulnerable than others.

2. New Immigrants or Non-Native Language Speakers

For this group of users, SMS messages are more likely to appear official or trustworthy, especially when written in a language they do not fully understand. New immigrants often expect to receive messages from government offices, banks, and other official institutions, which can increase their anxiety about missing important information and make them more inclined to interact with unfamiliar links or requests.

3. People Expecting Deliveries

People who are waiting for packages are more vulnerable to phishing SMS messages. When someone expects a delivery, they often receive real updates from companies like DHL, UPS, or Israel Post. Because of this, a fake message asking them to “confirm delivery details” or “pay a small fee” can look completely legitimate.

4. Small Business Owners / Freelancers

Small business owners and freelancers receive numerous legitimate messages every day from clients, suppliers, delivery services, payment providers, and government offices. Because their work often depends on quick responses and constant communication, it becomes harder for them to distinguish between real operational messages and fraudulent ones.

5. Young adults and teenagers

Although this type of user is heavy mobile users, they tend to be more impulsive and to click quickly without thinking deeply about the risks.

2. Literature Review and Research

2.1 Literature Review

We reviewed five academic articles related to SMS phishing detection and machine learning based classification systems. The following section summarizes the main findings from this literature review.

All of the reviewed articles highlighted machine learning and deep learning techniques as the primary approaches for detecting SMS phishing attacks. Most studies focus on analyzing message content, URLs, phone numbers, and behavioral indicators in order to distinguish legitimate messages from malicious ones.

Traditional machine learning approaches commonly rely on preprocessing techniques such as tokenization, stemming, normalization, and TF-IDF feature extraction. More recent studies increasingly employ advanced Natural Language Processing (NLP) techniques and transformer-based models, such as BERT, to better understand the contextual meaning of SMS messages.

The reviewed literature demonstrates that combining multiple detection techniques often improves classification accuracy. Some studies utilize ensemble learning methods, while others apply hybrid deep learning architectures to capture both local textual patterns and broader contextual relationships within messages.

In addition, the studies emphasize the importance of balanced datasets and large collections of real-world SMS messages for improving model performance and achieving more reliable results.

Despite the promising results reported in this research, several limitations were identified. Many studies validate their models only on static datasets and do not evaluate them in real time environments or within actual mobile applications.

Furthermore, several papers provide only limited analysis of false positives and false negatives, making it difficult to fully assess the weaknesses of the proposed approaches. Some review papers are primarily theoretical and do not include implementation details, datasets, or experimental validation.

As a result, it is often difficult to determine whether the reported performance metrics truly reflect real world effectiveness or are influenced by overfitting to the evaluation dataset.

Overall, the literature suggests that modern NLP and deep learning approaches, particularly transformer based models such as BERT, offer significant advantages over traditional keyword-based detection systems. This was one of the key conclusions that influenced the design of our proposed solution.

2.2 Market Research

We found several existing solutions that already attempt to address the growing problem of Smishing, but each still suffers from significant limitations. We will provide a small overview of each.

1. [ScanMySMS](#)

A website phishing detection platform that allows users to manually paste suspicious SMS messages into the website to receive an AI based analysis. While it provides accessible phishing detection without installing an app, it lacks real time and on device protection because users must manually copy each message into the system.

This solution only works if user suspects the message and won't prevent scenarios where user believes this message is legitimate.

2. [AI Smishing Checker iOS application](#)

IOS application which also uses AI to analysis and evaluates suspicious SMS messages and any text messages (could be emails for example) by allowing users to paste messages into the app for inspection.

The application focuses on detecting fraud using contextual AI analysis and provides recommendations regarding message safety.

However, like ScanMySMS, it still depends heavily on manual interaction from the user and does not provide passive and real time SMS monitoring.

3. [Apple's SMS filtering system](#)

Built in IOS filtering system inside the SMS application.

Automatically separates messages from unknown senders and may classify some content as spam.

In newer iOS versions, Apple also introduced additional filtering categories such as promotions and transactions.

However, these advanced filtering features are limited in availability across countries (currently only available in Brazil and India), and the system does not explicitly distinguish between phishing and general spam. Furthermore, all filtered messages are hidden from user and don't without sending a

notification, which can become problematic in cases of false positives where legitimate messages are incorrectly classified.

Overall, although these solutions contribute to improving SMS security, they still rely largely on user awareness and manual interaction rather than offering fully automated, intelligent, real time phishing protection directly on the device.

3. Proposed Solution

3.1 System Requirements

3.1.1 Functional Requirements

1. SMS Integration – The system shall connect to and monitor incoming SMS messages on the user's device.
2. Message Classification – The system shall classify incoming SMS messages as either phishing or legitimate.
3. Real-Time Analysis – The system shall automatically analyze SMS messages in real time as they are received.
4. Threat Notification – The system shall immediately notify the user when a suspicious or phishing message is detected.
5. User Feedback – The system shall allow the user to provide feedback on classifications through notification actions.

3.1.2 Non-Functional Requirements

1. Usability – The system shall provide a clear and simple user interface.
2. Accuracy – The system shall achieve high classification accuracy with a low false positive rate.
3. Performance – The system shall perform real-time SMS analysis.
4. Reliability – The system shall maintain high reliability and stability during operation.
5. Compatibility – The system shall support older Android devices and multiple languages.
6. Security – The system shall ensure user privacy and secure handling of data

3.2 Solution Overview

Our proposed solution consists of three main components that work together to provide real-time smishing detection for SMS messages.

The first component is a machine learning classification model responsible for determining whether an incoming SMS message is legitimate or malicious. The second component is a Python backend server that hosts the trained model and exposes prediction services through a REST API. The third component is an Android application that monitors incoming SMS messages, communicates with the backend server, and notifies users when suspicious messages are detected.

When a new SMS message arrives on the user's device, the Android application automatically sends the message content to the backend server. The server forwards the request to the machine learning model, which analyzes the message and returns a prediction result. Based on the classification received, the Android application may display a warning notification to the user.

Together, these components provide an automated and near real-time solution for detecting smishing attempts on Android devices.

3.3 Solution Architecture

Our proposed solution follows a Client-Server pattern.

In a Client-Server architecture, clients are responsible for interacting with users and sending requests, while a central server processes those requests and returns the required results. This architecture separates the presentation layer from the business logic and processing components.

We selected this architecture for several reasons.

First, our ML model requires performing complex calculations when receiving a new message. Running those calculations on the user mobile device may affect performance and consume a lot of battery. In addition, hosting the model on a centralized server allows all users to benefit from the same trained model without requiring local model installation or execution.

Second, this architecture enables easier maintenance and future updates. Improvements to the machine learning model can be deployed on the server without requiring users to update the mobile application.

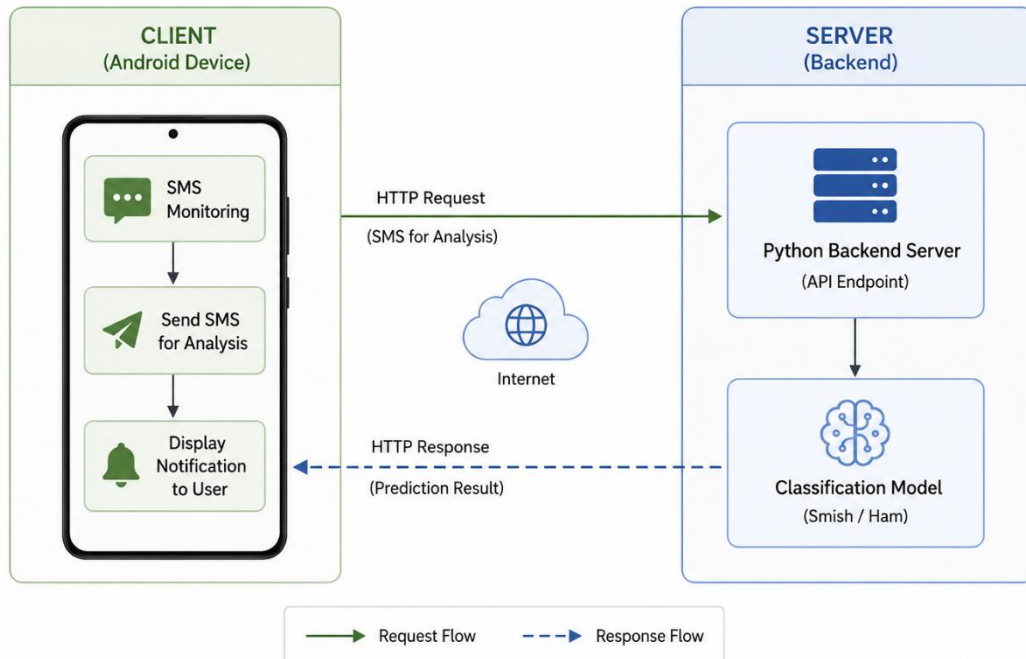
Third, the architecture improves scalability by allowing multiple Android devices to access the same backend service simultaneously.

In the proposed system, the Android application acts as the client. Its responsibility is to monitor incoming SMS messages, send them to the backend server for analysis, and present the results to the user.

The Python backend acts as the server component. It receives classification requests from Android devices, executes the machine learning model, and returns prediction results.

The machine learning model is hosted on the server and serves as the decision maker component of the system.

High-Level Client-Server Architecture

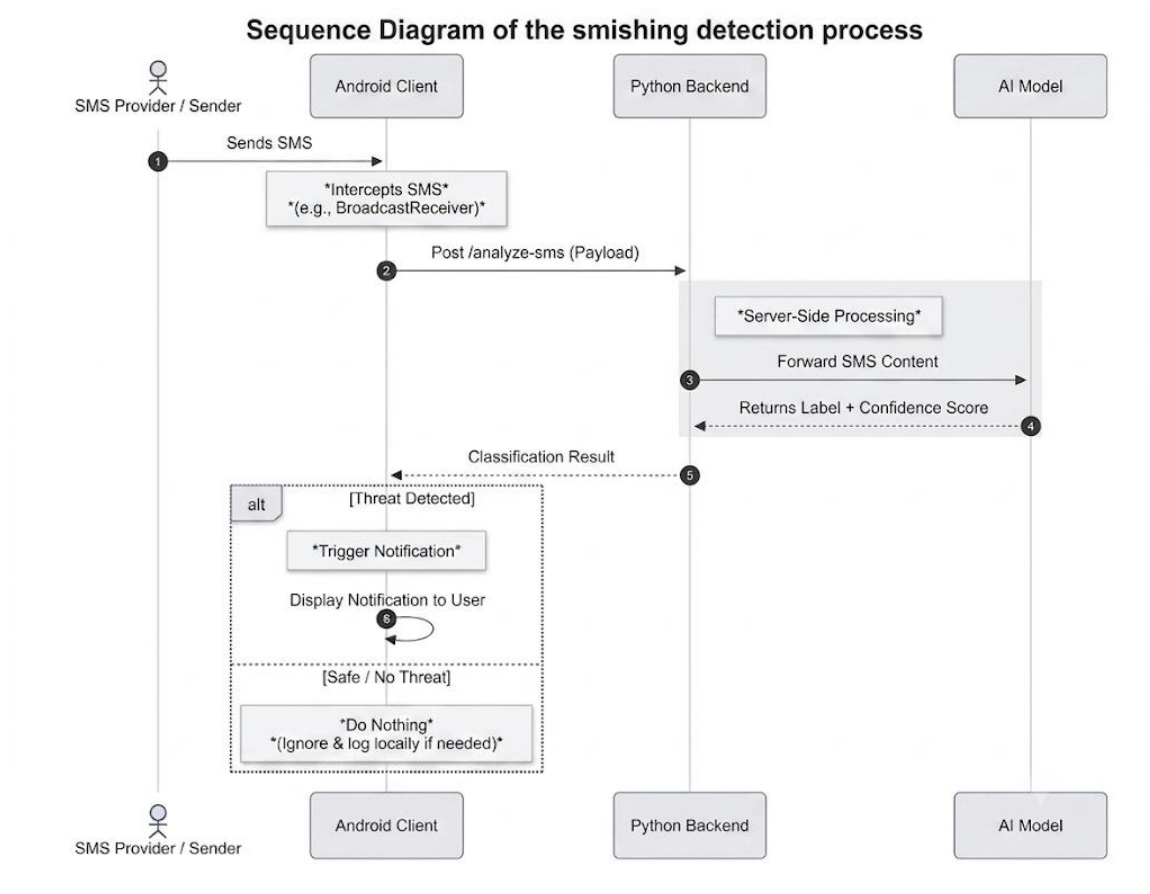


3.4 System Data Flow

The system operates according to the following workflow:

1. An SMS message arrives on the user's Android device.
2. The Android application intercepts the incoming message.
3. The message content is sent to the backend server through a REST API request.
4. The backend server forwards the message to the machine learning classification model.
5. The model analyzes the message and generates a classification result together with a confidence score.
6. The backend server returns the prediction result to the Android application.

7. If the message is classified as a smishing attempt, the application displays a warning notification to the user.
8. The classification result is presented to the user through the application interface.



4. Dataset

4.1 Dataset Sources

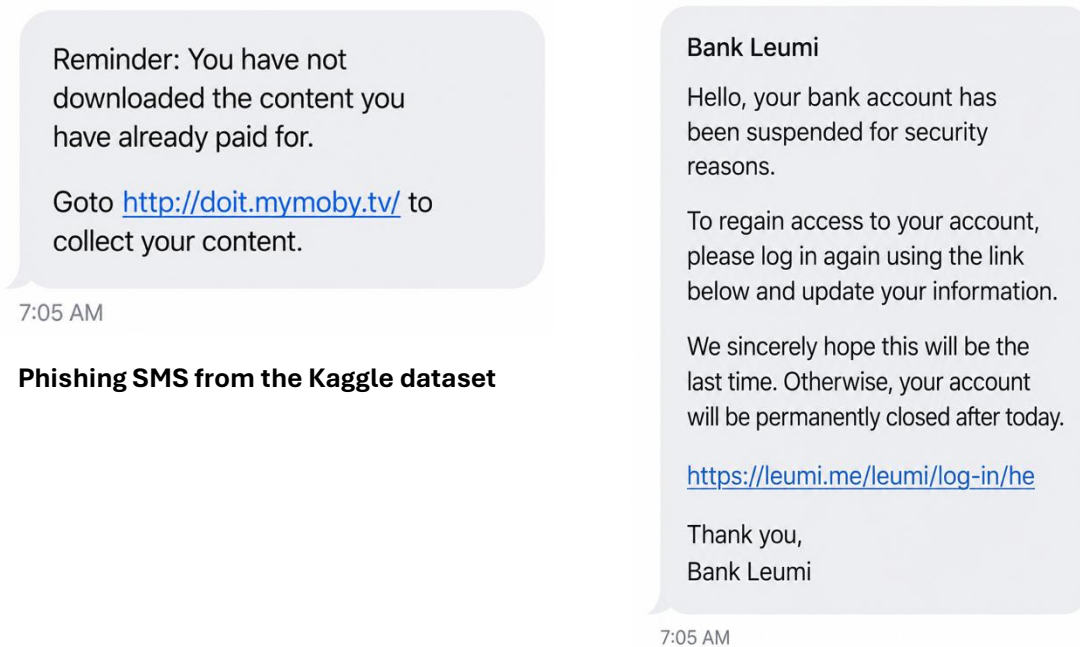
Our dataset used to train our ML model was built in several stages.

At the beginning of the project, we searched for publicly available datasets containing real world SMS messages, especially realistic phishing messages and complex legitimate messages. However, we found that such datasets are very limited and difficult to obtain. As a result, we started with two public datasets from Kaggle containing regular SMS messages together with basic spam and phishing messages.

These datasets provided a useful starting point, but during development we found that they did not fully capture the complexity of real world SMS messages. Many of

the phishing messages in these datasets were relatively simple, while real phishing attempts often contain brand impersonation, links, financial information, urgent language, and convincing calls to action. At the same time, many legitimate messages, such as bank notifications, payment confirmations, delivery updates, and service messages, also contain links and requests to perform some action, making them appear suspicious at first glance.

Examples of these differences are shown below. A phishing SMS taken from the Kaggle dataset is compared with a real-world phishing SMS. The examples highlight the differences in message length, wording, use of trusted brands, and the presence of more realistic and convincing links.



To overcome this gap, we collected examples of real SMS messages from various sources, including both legitimate messages and real phishing attempts. These examples were added to the dataset, but they also served as references for the level of sophistication found in real world messages. Rather than generating simple variations of existing messages, we used AI to learn the style, structure, and common patterns of these examples to create new messages with a similar level of complexity.

The goal was not to produce messages that were almost identical to the originals with only one or two words changed. Instead, AI generated entirely new messages with different wording, scenarios, and levels of urgency and impersonation. The included

included both complex legitimate messages containing real links to well-known websites and instructions for the user, as well as sophisticated phishing messages containing fake links, different impersonation scenarios, and wording that closely resembles real phishing attempts.

This approach allowed us to create a richer and more challenging dataset that better reflects the kinds of messages the model is expected to encounter in practice.

Finally, all data sources were merged into a single unified dataset, including the original Kaggle datasets, real world SMS examples collected during the project, and AI generated messages. This dataset was subsequently used for both model training and evaluation.

4.2 Dataset Structure

Each row in the dataset represents a single SMS message. In addition to the message text and its class label, the dataset also contains information about whether the message includes a URL, an email address, or a phone number. These fields provide useful information for the later stages of feature extraction and model training.

The label column is the target variable and indicates whether the message is a legitimate SMS (ham) or a phishing SMS (smish). The message column contains the raw SMS text. The remaining columns are binary indicators showing whether the message contains a URL, an email address, or a phone number.

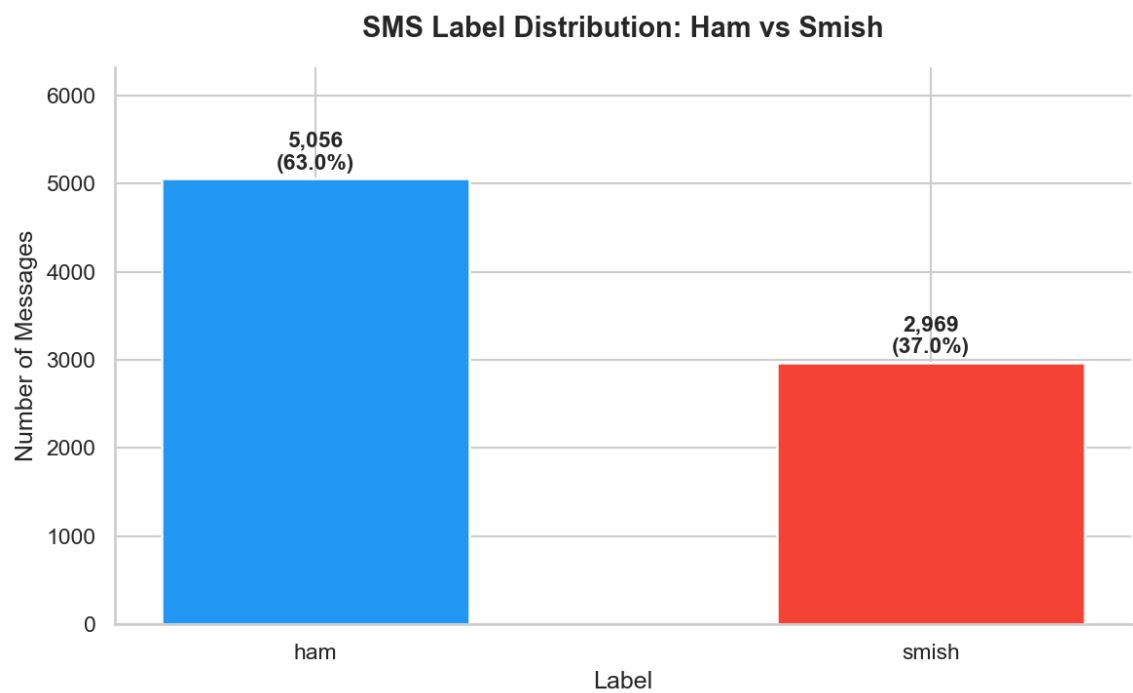
Column	Type	Meaning
label	categorical	Indicates whether the message is legitimate (ham) or phishing (smish). This is the target variable used for classification.
message	text	Contains the raw SMS message text.
url	binary (yes/no)	Indicates whether the message contains a URL.
email	binary (yes/no)	Indicates whether the message contains an email address.
phone	binary (yes/no)	Indicates whether the message contains a phone number or short code.

Before proceeding with preprocessing and model training, a consistency check was performed to verify the integrity of the dataset. In particular, the dataset was inspected for missing values and the data types of all columns were examined. No missing values were found, and all columns were loaded successfully. These checks confirmed that the dataset was complete and ready for the subsequent preprocessing and feature extraction stages.

4.3 Dataset Distribution

After combining the different data sources, the final dataset contained 8,025 SMS messages. Among them, 5,056 messages were labeled as legitimate (ham), while 2,969 messages were labeled as phishing (smish).

Class	Number of Messages	Percentage
Ham	5,056	63%
Smish	2,969	37%
Total	8,025	100%



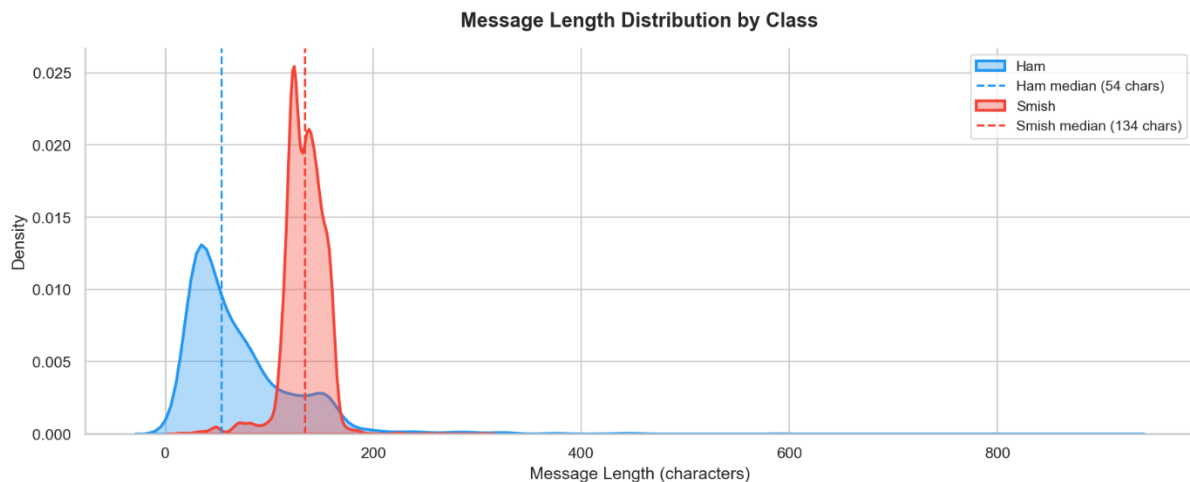
Although the dataset is not perfectly balanced, the imbalance is relatively moderate and reflects real world conditions, where legitimate messages are more common than phishing messages. Therefore, the dataset still contains enough examples from both classes and allows the models to learn meaningful patterns.

Beyond the overall class distribution, additional analyses revealed several important differences between legitimate and phishing messages. These differences later motivated the features of engineering process described in Chapter 5.

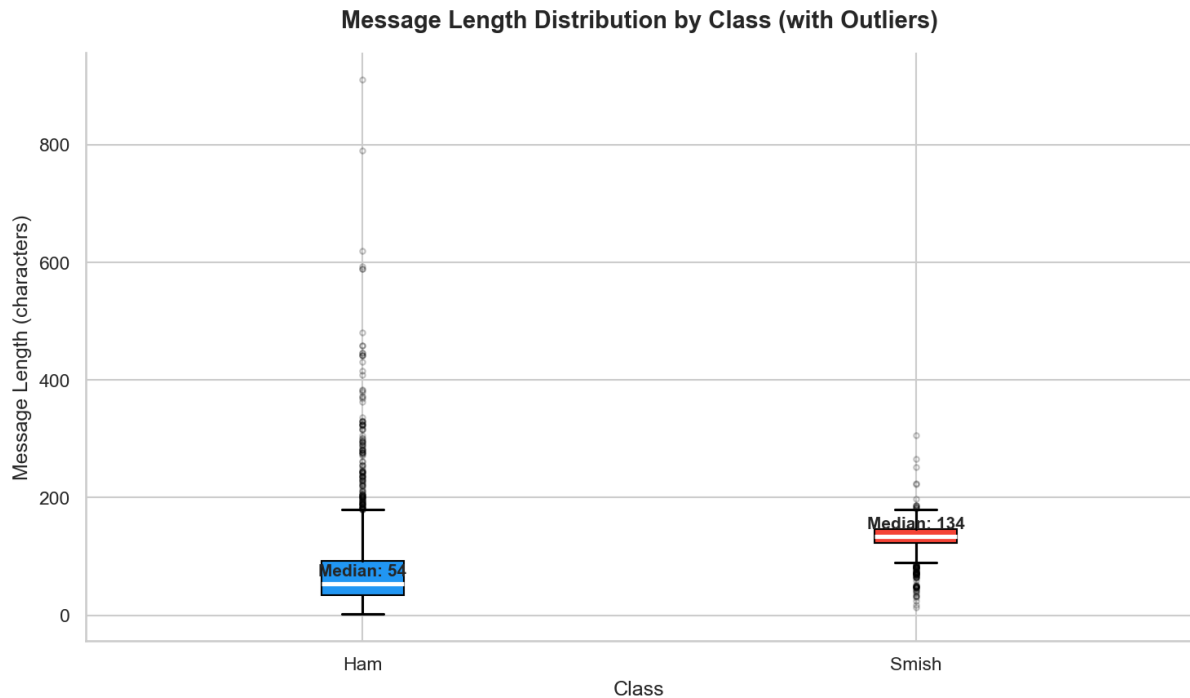
Message Length

To better understand the differences between the two classes, the lengths of the messages were analyzed.

Overall, phishing messages tend to be longer than legitimate messages. Many phishing messages contain explanations, warnings, requests for immediate action, impersonation of trusted organizations, and links to fraudulent websites. In contrast, legitimate messages are generally shorter and more concise.



The analysis also revealed that phishing messages exhibit a higher median length and greater variability, with several unusually long messages appearing mainly in this class. These observations suggest that message length may provide useful information for distinguishing between legitimate and phishing SMS messages.

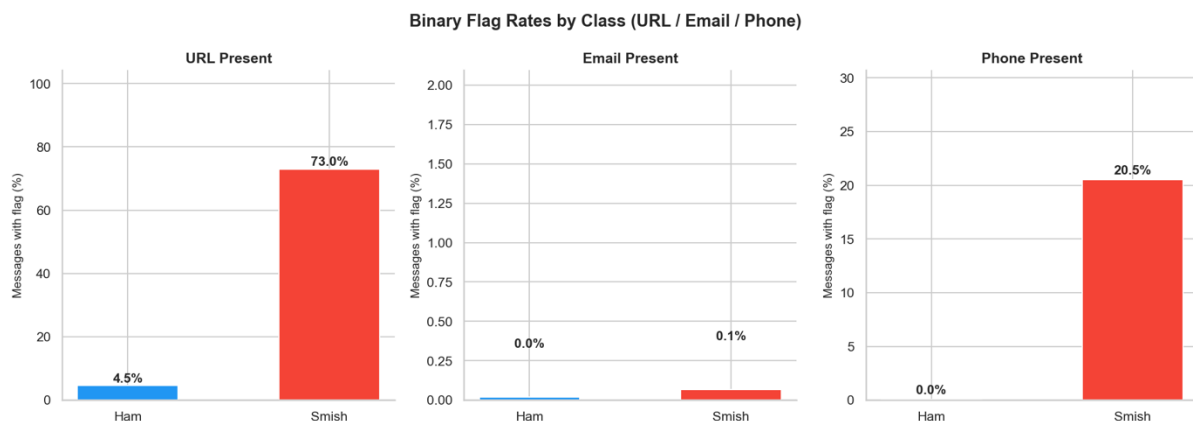


Presence of URLs, Email Addresses, and Phone Numbers

Another important difference between the two classes is the presence of additional contact information.

URLs appear much more frequently in phishing messages than in legitimate messages, making them an important indicator for detecting smishing attacks. In contrast, phone numbers and email addresses are relatively uncommon and provide less separation between the two classes.

These results indicate that the existence of a URL is much more informative than the presence of an email address or a phone number.

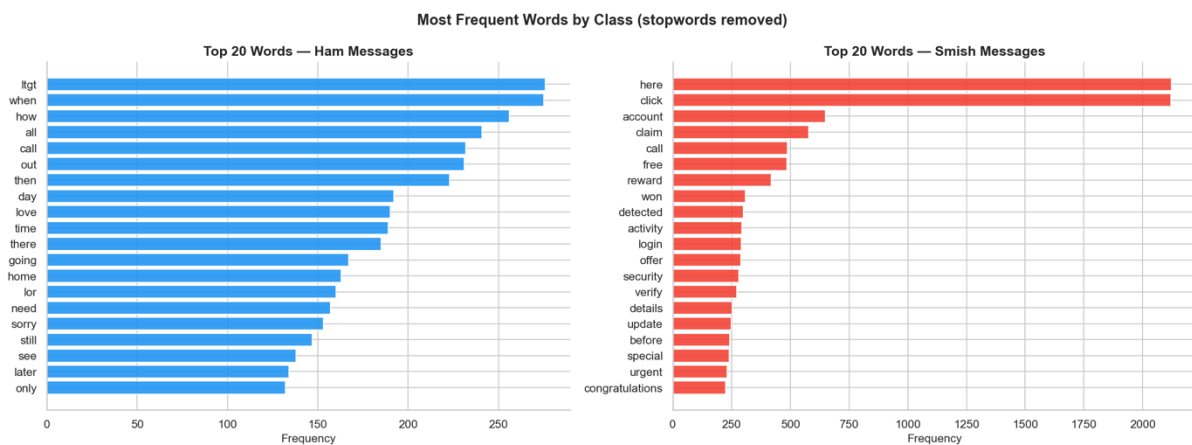


Vocabulary Analysis

The most common words used by legitimate and phishing messages were also examined.

Legitimate messages are mainly characterized by common conversational words, while phishing messages contain more action oriented and urgent expressions. These differences reflect the different goals of the two classes: legitimate messages are usually intended to provide information, whereas phishing messages attempt to encourage users to perform immediate actions.

Therefore, the textual content itself contains valuable information that can be exploited by natural language processing techniques.



4.4 Data Cleaning & Preprocessing

Before extracting features and training the machine learning models, several preprocessing steps were performed to improve the consistency of the dataset and address issues discovered during the exploratory analysis.

Text Normalization

SMS messages often contain specific URLs, account numbers, transaction amounts, and other numerical values. If these values are kept unchanged, the models may treat each one as a completely different word, making it harder to learn meaningful patterns.

To reduce unnecessary variations, a text normalization process was applied to all messages. URLs belonging to known and trusted organizations, such as banks,

government services, and well known companies, were replaced with a dedicated `<LEGIT_URL>` token, while all other links were replaced with a general `<URL>` token. In addition, standalone sequences of three or more digits were replaced with the `<NUM>` token.

This preprocessing step prevented the models from memorizing specific URLs or reference numbers and allowed them to focus on the overall structure and meaning of the messages instead. Such techniques are commonly used in Natural Language Processing (NLP) applications and help machine learning models generalize better to previously unseen data.

```
examples = [
    "Bank Hapoalim: Your account ending in 3847 was charged 11.50 NIS. View: https://www.bankhapoalim.co.il",
    "URGENT: verify your account now at http://bit.ly/Xy93kP or it will be suspended!",
]
for ex in examples:
    print("RAW :", ex)
    print("CLEAN:", preprocess_text(ex))
    print()
```

Python

```
RAW : Bank Hapoalim: Your account ending in 3847 was charged 11.50 NIS. View: https://www.bankhapoalim.co.il
CLEAN: bank hapoalim: your account ending in <num> was charged 11.50 nis. view: <legit_url>

RAW : URGENT: verify your account now at http://bit.ly/Xy93kP or it will be suspended!
CLEAN: urgent: verify your account now at <url> or it will be suspended!
```

Addressing Hard-Negative Messages

Although the overall dataset distribution was relatively balanced, further analysis revealed a much stronger imbalance within a specific subset of messages. Legitimate messages containing URLs and financial information were extremely rare compared with phishing messages exhibiting the same characteristics.

As a result, the models could incorrectly learn that the combination of a URL and a financial amount is itself an indication of phishing. Such behavior would lead to false alarms for real banking notifications, delivery updates, OTP codes, insurance statements, and investment reports.

To address this issue, 116 realistic legitimate messages were manually created and added to the dataset. These messages represented scenarios that are commonly encountered by users, including Israeli and international banks, insurance and pension notifications, government services, one-time passwords, delivery confirmations, and investment statements.

After the augmentation process, the number of legitimate messages increased from 5,056 to 5,172, while the number of phishing messages remained unchanged at 2,969.

Class	Before	After
Ham	5,056	5,172
Smish	2,969	2,969
Total	8,025	8,141

Effect of the Augmentation

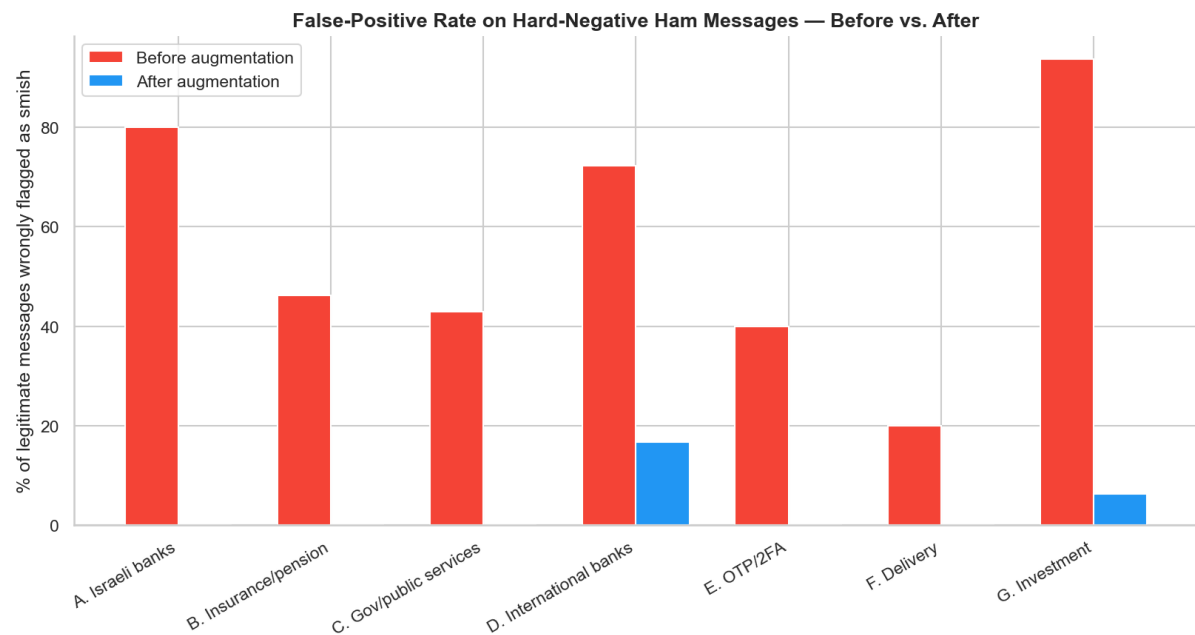
To evaluate the impact of the augmentation process, a model trained on the original dataset was compared with a model trained on the augmented dataset.

Before the augmentation, the model incorrectly classified a large proportion of legitimate financial messages as phishing. In particular, Israeli bank notifications, investment reports, insurance messages, and government notifications suffered from high false-positive rates because they contained characteristics commonly associated with phishing attacks, such as URLs and numerical values.

After adding the additional examples, the false positive rate decreased substantially across all categories. The improvement was especially noticeable for banking and investment messages, demonstrating that the models learned to distinguish between suspicious content and legitimate financial communication rather than simply relying on superficial characteristics.

Category	Number of Messages	False Positive Rate Before (%)	False Positive Rate After (%)
A. Israeli Banks	25	80.0	0.0
B. Insurance / Pension	13	46.2	0.0
C. Government / Public Services	14	42.9	0.0

Category	Number of Messages	False Positive Rate Before (%)	False Positive Rate After (%)
D. International Banks	18	72.2	16.7
E. OTP / 2FA	15	40.0	0.0
F. Delivery	15	20.0	0.0
G. Investment	16	93.8	6.2



5. Feature Engineering

Machine learning algorithms cannot process raw text directly. Therefore, before training the classification models, each SMS message had to be transformed into a numerical representation.

This process, commonly referred to as feature engineering, aims to extract meaningful information from the messages and represent it in a form that machine learning algorithms can analyze.

Different types of features capture different aspects of a message. Some features focus on the importance of specific words and phrases, while others capture the overall meaning of the text. In addition, several handcrafted features were incorporated to describe characteristics such as the presence of URLs and phone numbers.

Combining these complementary sources of information allowed the models to utilize both lexical and semantic characteristics of SMS messages. The following sections describe each component and explain the motivation behind its inclusion.

5.1 TF-IDF Features

One of the most widely used techniques in Natural Language Processing (NLP) is Term Frequency Inverse Document Frequency (TF-IDF). This method assigns a numerical importance score to each word according to two factors: how frequently the word appears within a particular message and how rare it is across the entire dataset. As a result, common words receive lower importance, while more distinctive words receive higher weights.

TF-IDF captures exact lexical patterns and highlights characteristic words and phrases that may be useful for distinguishing phishing messages from legitimate ones. For example, phishing messages often contain urgent requests, account related terminology, and references to financial services. Such patterns can provide valuable information for the classification process.

Before extracting TF-IDF features, all messages were normalized using the preprocessing procedure described in Section 4.4. This prevented the model from memorizing specific URLs, reference numbers, or monetary amounts and encouraged it to learn more general textual patterns.

To increase the expressive power of the representation, both individual words and short two-word expressions were considered. In addition, only the most informative terms were retained in order to avoid introducing unnecessary noise into the feature vector.

Finally, the TF-IDF component contributed 50 lexical features, which were later combined with the other feature groups described in the following sections.

The selected terms illustrate the types of lexical patterns captured by the TF-IDF representation. Many of the retained words and phrases are characteristic of phishing campaigns and include terms related to account verification, rewards, urgency, and online services. Examples include "verify", "urgent", "reward", "secure", and "account", as well as two word expressions such as "verify account", "claim reward", and "activate reward". These patterns provide additional information that can help distinguish phishing messages from legitimate ones.

Category	Example Terms
Account and security	account, verify, secure, login
Urgency and action	urgent, action, update
Rewards and prizes	reward, prize, won, congratulations
Online services	amazon, netflix
Two word expressions	verify account, claim reward, activate reward

5.2 SBERT Embeddings

While TF-IDF focuses on exact words and phrases, it does not capture the overall meaning of a message. Therefore, a semantic representation based on Sentence-BERT (SBERT) was also incorporated.

SBERT is a language model derived from Bidirectional Encoder Representations from Transformers (BERT), a deep learning architecture that has become widely used in Natural Language Processing (NLP). Unlike traditional word based representations, SBERT converts an entire sentence into a dense numerical vector, commonly referred to as an embedding. Messages with similar meanings tend to have similar embeddings, even when they use different words or sentence structures.

This property is particularly useful for smishing detection, since phishing campaigns often express the same intent using different wording. As a result, relying solely on exact keywords may limit the ability of the model to generalize to previously unseen messages.

In this project, each message was encoded using the pre trained all-MiniLM-L6-v2 SBERT model. The resulting embedding consisted of 384 numerical features representing the semantic content of the message. These features were later combined with the TF-IDF representation and the additional engineered features described in the following sections.

To illustrate the semantic capabilities of SBERT, cosine similarity was computed between several message embeddings. Messages with similar meanings produced relatively high similarity scores, even when their wording differed, whereas unrelated messages resulted in much lower scores. This behavior demonstrates the ability of SBERT to capture semantic information rather than relying solely on exact lexical patterns.

Message Pair	Cosine Similarity
Two phishing messages with different wording	0.714
Phishing message and unrelated casual message	0.133

5.3 URL Features

Links are one of the most common characteristics of smishing attacks. Phishing messages often attempt to redirect victims to malicious websites in order to steal credentials, personal information, or financial details. Consequently, URL related features can provide valuable information for the classification process.

Three complementary URL features were incorporated in this project. The first feature indicates whether a message contains a URL. Although legitimate messages may also include links, the presence of a URL is generally more common in phishing messages.

The second feature focuses on shortened URLs. URL shortening services are frequently used in smishing campaigns because they conceal the true destination of the link and make it more difficult for users to recognize suspicious domains.

The third feature detects suspicious domain patterns that may indicate brand impersonation. Attackers often construct domains that resemble well known organizations by combining trusted brand names with words such as "secure", "login", or "account". Examples include domains designed to imitate financial institutions or popular online services.

These features were extracted using pattern matching techniques. The first detector searched for strings that resemble web addresses in order to identify the presence of URLs. A second detector examined whether the link belonged to one of several well known URL shortening services. Finally, the domain names themselves were analyzed for suspicious patterns commonly associated with brand impersonation and phishing campaigns.

By combining these complementary detection mechanisms, the system was able to capture different levels of URL related information, ranging from the mere presence of a link to more sophisticated indicators of malicious intent.

These complementary features allow the model to distinguish between ordinary links and URLs that exhibit characteristics commonly associated with phishing attacks.

Example URL	Description
https://www.dhl.com/tracking	Legitimate URL
http://bit.ly/Xy93kP	Shortened URL
http://paypal-secure-login7.net	Suspicious domain impersonating a trusted brand

5.4 Phone Number Features

Phone numbers and SMS short codes are commonly used in both legitimate and phishing messages. For example, organizations may provide customer service numbers, while attackers may encourage victims to reply to premium rate numbers or contact fraudulent services. Consequently, the presence of phone related information may provide additional clues for the classification process.

A dedicated feature was therefore incorporated to identify phone numbers and SMS short codes within messages. Conventional phone number formats were considered, including numbers written with country codes or separators. In addition, sequences of five or more consecutive digits were also treated as phone-related information, allowing the system to detect SMS short codes that may not resemble traditional phone numbers.

These patterns were identified using pattern matching techniques designed to capture different representations of phone numbers. As a result, the feature was able to recognize both standard telephone numbers and shorter numeric codes frequently used in SMS communications.

Example Message	Phone Feature
"Call us at 03-6375525 for details"	1
"Reply YES to 87121 to confirm"	1
"See you at 8"	0

5.5 Email Features

Email addresses occasionally appear in SMS messages and may be used by both legitimate organizations and phishing campaigns. Therefore, an additional feature was initially introduced to identify the presence of email addresses within messages.

Email addresses were detected using pattern-matching techniques designed to recognize common email formats. This allowed the system to distinguish between messages containing email addresses and messages that did not.

However, further analysis revealed that email addresses are relatively uncommon in SMS messages in general. Consequently, the presence of an email address provided very little discriminatory information for distinguishing between legitimate and phishing messages.

As a result, although the email feature continued to be computed and remained available for diagnostic purposes, **it was excluded from the final feature vector used to train the classification models.** This decision reduced unnecessary complexity without sacrificing classification performance.

Example Message	Email Feature
“Contact support@bank-helpdesk.com to resolve this”	1
“See you tomorrow!”	0

5.6 Suspicious Keywords Features

Many phishing messages attempt to persuade users to visit fraudulent websites by encouraging them to perform specific actions. Typical examples include requests to log in, verify an account, confirm personal information, or reactivate a service. Such expressions are commonly used to direct victims toward fake login pages and credential theft attempts.

To capture these patterns, a dedicated feature was introduced to identify phrases associated with authentication and account verification. Unlike general urgency related features, which focus on the tone of the message, this feature specifically targets the call-to-action itself.

These patterns were detected using pattern matching techniques based on common expressions frequently observed in phishing messages. As a result, the feature was able to distinguish messages containing account-related requests from ordinary conversational messages.

Example Message	Suspicious Keywords Feature
“Please log in to verify your account immediately”	1
“Don't forget the meeting at 3 pm”	0

5.7 Urgency and Threat Features

A common characteristic of phishing messages is the use of psychological pressure to influence victims and encourage immediate action. Attackers often attempt to create a sense of urgency, fear, or concern in order to reduce the likelihood that users will carefully examine the message before responding.

To capture these behavioral patterns, three complementary features were incorporated. The first feature identifies expressions associated with urgency, such as requests for immediate action or warnings about expiring opportunities. The second feature focuses on threatening language related to account suspension, unusual activity, or other security concerns. The third feature detects explicit requests to follow links or perform specific actions.

These features were extracted using pattern matching techniques based on expressions commonly observed in phishing campaigns. By separating urgency, threat, and action requests into individual features, the model was able to capture different forms of emotional pressure frequently used by attackers.

Unlike the suspicious keywords feature described in Section 5.6, which focuses on account-related actions such as logging in or verifying information, the features described in this section concentrate on the emotional tone of the message and the pressure applied to the recipient.

Example Message	Urgency	Threat	Action Request
“URGENT: unusual activity detected, your account has been suspended. Click here to restore access.”	1	1	1
“Hey, running 10 minutes late, see you soon!”	0	0	0

5.8 Message Structure Features

In addition to lexical and semantic information, several features were introduced to capture the structural characteristics of SMS messages. Unlike the previous features, which focus on specific words or phrases, these features describe how a message is written regardless of its exact content.

Three complementary structural features were incorporated. The first feature, `avg_word_length`, measures the average length of words within a message. Phishing messages often contain longer words due to the presence of URLs, reference numbers, and more formal language.

The second feature, `digit_count`, counts the number of digits appearing in the message. This feature captures patterns related to monetary amounts, account numbers, reference codes, and one-time passwords.

The third feature, `special_char_count`, measures the number of special characters appearing in the message. Symbols such as exclamation marks and characters associated with financial transactions are frequently used to emphasize urgency or attract the recipient's attention.

These features were computed directly from the message text and provide additional information that complements the lexical and semantic representations described in the previous sections.

Statistical analysis of the dataset revealed noticeable differences between legitimate and phishing messages. In particular, phishing messages exhibited larger values of both `avg_word_length` and `digit_count`. On average, phishing messages contained approximately eleven times more digits than legitimate messages, reflecting the frequent presence of amounts, account identifiers, reference numbers, and OTP style codes.

These observations suggest that structural characteristics can provide useful information for distinguishing between legitimate and phishing messages.

```
Class means – structural features:
      avg_word_length  digit_count  special_char_count
label
ham                5.296         0.546             0.283
smish              7.619         6.143             0.634
```

5.9 Additional Custom Features

In addition to the features described in the previous sections, several custom features were introduced to capture patterns that are not directly represented by keywords or message structure.

Two features, `has_specific_reference_number` and `contains_official_domain`, were designed to identify characteristics commonly found in legitimate messages. Reference numbers and case identifiers are frequently used by trusted organizations, while links pointing to official domains, such as government websites or well-known banks, provide another indication that a message may be legitimate.

Another feature, `has_exact_amount`, was used to detect precisely formatted monetary amounts. Such amounts may appear in both legitimate messages and phishing messages. Therefore, this feature does not strongly indicate either class and serves mainly as additional information.

Finally, two aggregate features, `smish_signal_count` and `ham_signal_count`, were introduced. These features count the number of suspicious and legitimate indicators identified within a message. Considering several signals together can provide more information than relying on a single feature, since messages often exhibit multiple related patterns at the same time.

Statistical analysis of the dataset revealed that `smish_signal_count` provides a particularly strong distinction between the two classes. On average, legitimate messages had a value of only 0.014, whereas phishing messages had an average value of 1.336. This suggests that the accumulation of several suspicious indicators can provide valuable information for smishing detection.

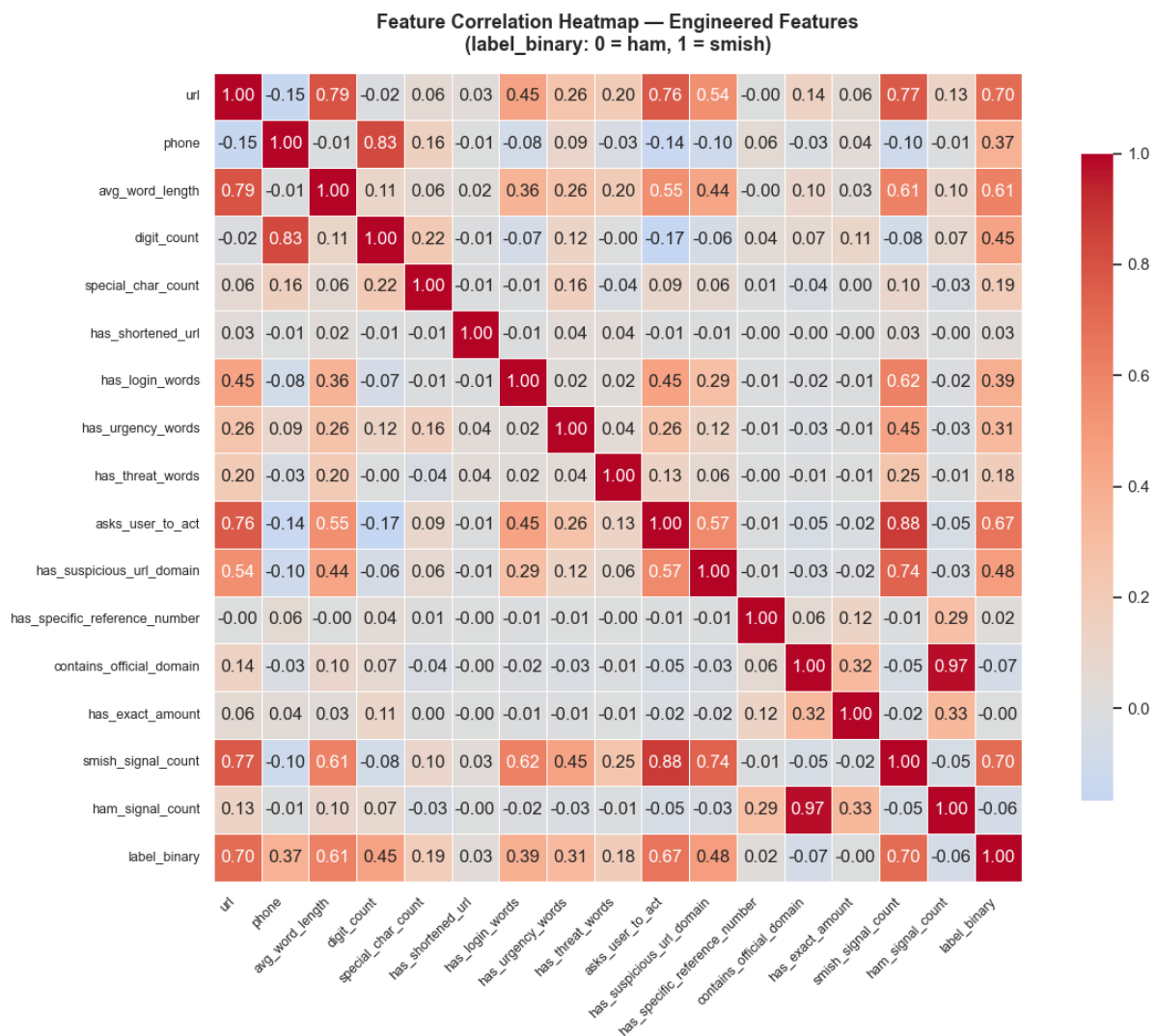
```
Class means – custom + aggregate features:
      has_specific_reference_number  contains_official_domain  \
label
ham                                0.000                      0.013
smish                             0.001                      0.000

      has_exact_amount  smish_signal_count  ham_signal_count
label
ham                0.002                0.014                0.014
smish              0.002                1.336                0.001
```

5.10 Feature Importance Analysis

Before examining the importance assigned by the trained models, it is useful to first investigate the relationships between the engineered features and the target class. Correlation analysis provides a simple way to evaluate how strongly each feature is associated with legitimate and phishing messages, independently of any specific classifier.

Positive correlation values indicate that a feature tends to appear more frequently in phishing messages, whereas negative values indicate an association with legitimate messages. Features with values close to zero exhibit little direct relationship with the target class.



The correlation analysis revealed that features such as `smish_signal_count`, `url`, `asks_user_to_act`, and `avg_word_length` exhibit relatively strong positive associations with the phishing class. In contrast, `ham_signal_count` and `contains_official_domain` show weak or negative associations, reflecting their relationship with legitimate messages.

Although correlation analysis provides useful insights, it evaluates each feature individually and does not account for interactions between features. Therefore, feature importance analysis was performed using the trained classification models in order to better understand which features contributed most to the final predictions.

For the tree based models, feature importance values were obtained directly from the models, while the absolute values of the coefficients were used for Logistic Regression. This analysis did not require retraining the models and was performed using the final saved classifier.

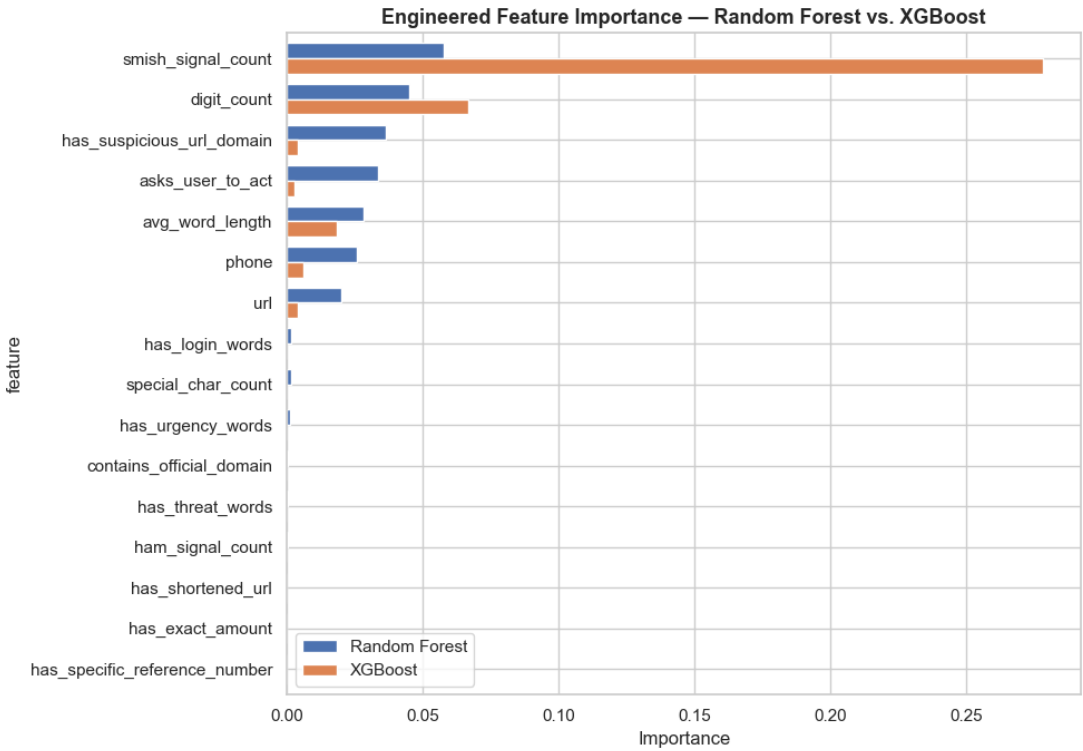
The results showed that the SBERT embeddings contributed most of the predictive information. Depending on the model, the SBERT component accounted for approximately 56-80% of the total importance. This behavior is expected, since the semantic embeddings capture the overall meaning of a message and represent the largest feature block.

The engineered text features also provided a significant contribution, particularly in XGBoost, where they accounted for approximately 37% of the total importance. In contrast, the binary features and TF-IDF features contributed smaller but still meaningful amounts of information.

Among the interpretable features, `smish_signal_count` consistently emerged as the most important feature across all models. This result indicates that the accumulation of multiple suspicious indicators provides stronger evidence than relying on a single pattern alone.

Other highly important features included `digit_count`, `has_suspicious_url_domain`, `asks_user_to_act`, `avg_word_length`, and the phone and url indicators. These findings suggest that the models rely primarily on combinations of suspicious patterns and structural properties of messages, rather than on isolated keywords.

	Random Forest	XGBoost	Logistic Regression
SBERT (384)	67.4	55.700001	79.7
flags (2)	4.6	1.000000	2.4
text features (14)	20.6	37.000000	6.3
TF-IDF (50)	7.4	6.300000	11.5



	feature	Random Forest	XGBoost	Logistic Regression (lcoefl)
0	smish_signal_count	0.057509	0.278452	0.851248
1	digit_count	0.045087	0.066808	1.500007
2	has_suspicious_url_domain	0.036389	0.003901	0.852180
3	asks_user_to_act	0.033522	0.002683	0.846457
4	avg_word_length	0.028134	0.018339	0.149735
5	phone	0.025894	0.006005	1.595016
6	url	0.019847	0.003787	0.727534
7	has_login_words	0.001513	0.000000	0.081926
8	special_char_count	0.001383	0.000000	0.059053
9	has_urgency_words	0.001081	0.000000	0.579963
10	contains_official_domain	0.000311	0.000000	0.313231
11	has_threat_words	0.000304	0.000000	0.257397
12	ham_signal_count	0.000255	0.000000	0.286698
13	has_shortened_url	0.000011	0.000000	0.163665
14	has_exact_amount	0.000009	0.000000	0.002365
15	has_specific_reference_number	0.000000	0.000000	0.060382

6. Machine Learning Model Development

6.1 Defining the Classification Problem

The task addressed in this study is a binary classification problem. Given the 450 dimensional feature vector extracted from an SMS message, the model predicts whether the message is legitimate (ham) or a smishing message (smish).

Two characteristics of the problem influence the design and evaluation of the models. First, the dataset is not perfectly balanced, with legitimate messages being more common than smishing messages. As a result, overall accuracy alone does not provide a complete picture of model performance. Additional evaluation measures are therefore required to assess how well the models identify both classes.

Second, the consequences of different types of errors are not equal. Failing to detect a smishing message may expose users to financial loss or credential theft, whereas incorrectly classifying a legitimate message mainly causes inconvenience. For this reason, special attention is given to evaluation measures that balance precision and recall and provide a more meaningful assessment of phishing detection performance.

6.2 Text Preprocessing

The preprocessing procedure applied to the SMS messages was described in detail in Section 4.4. Before feature extraction, each message undergoes the same normalization process in order to obtain a consistent representation.

Since this preprocessing stage follows fixed rules and does not depend on the training data, it can be applied uniformly to the training set, the test set, and any future incoming messages. As a result, all messages pass through the same initial stage before feature extraction and classification.

The remaining stages of the machine learning pipeline are described in the following section.

6.3 Text Representation

After preprocessing, each SMS message must be converted into a numerical representation that can be processed by the machine learning models. Since no single representation captures all aspects of a message, multiple feature groups are combined to provide complementary information.

6.3.1 TF-IDF Representation

The TF-IDF representation described in Section 5.1 captures exact lexical patterns and highlights informative words and short phrases. This representation is derived from the training messages and is then applied consistently to unseen messages, ensuring that the evaluation reflects the expected performance on future data.

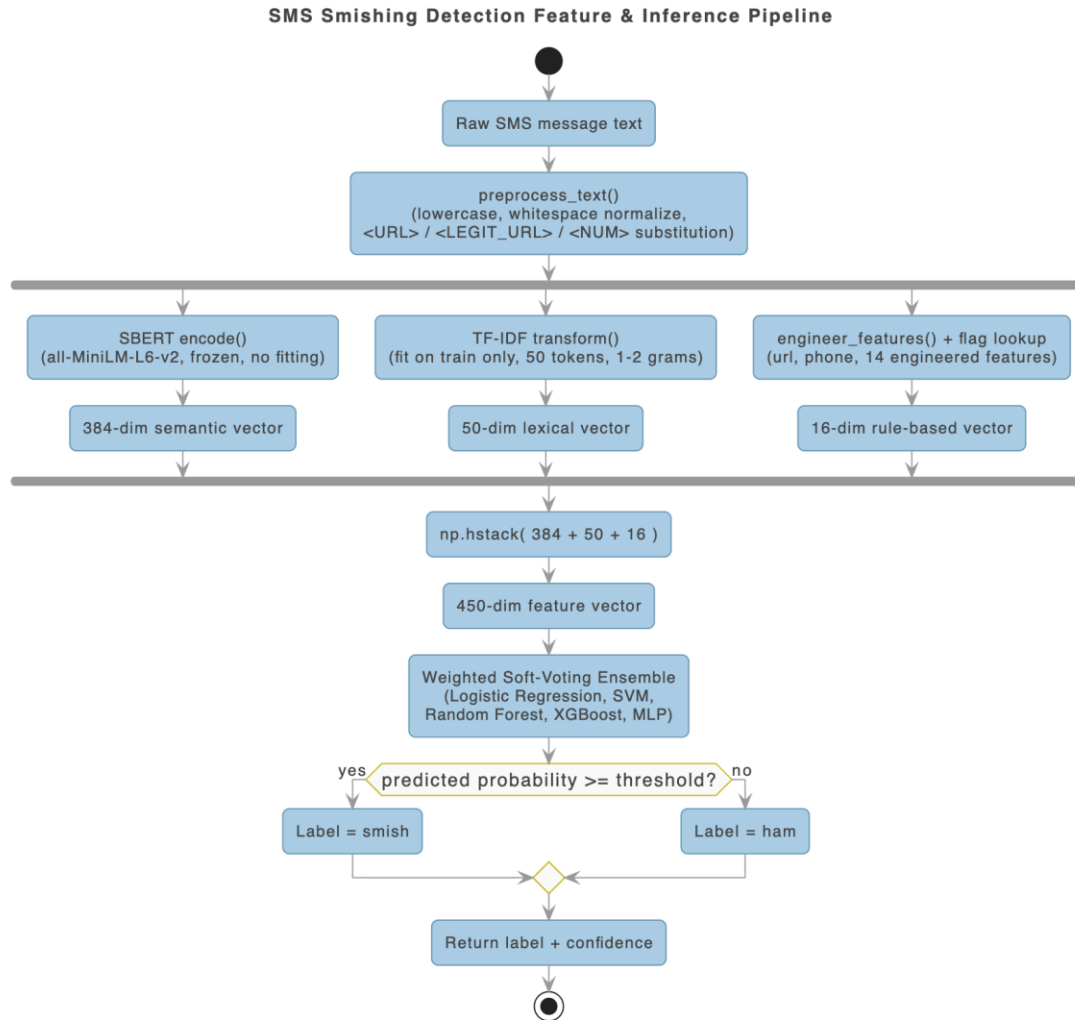
6.3.2 SBERT Representation

The SBERT representation introduced in Section 5.2 captures the semantic meaning of a message. Unlike TF-IDF, SBERT relies on a pre-trained language model and does not learn additional information from the dataset used in this project. Consequently, the same representation procedure can be applied uniformly to both training and test messages.

In addition to the lexical and semantic representations, the binary indicators and handcrafted features presented in Chapter 5 are incorporated into the final representation. These features provide information about suspicious links, message structure, account-related requests, urgency, and other patterns that may distinguish smishing messages from legitimate messages.

By combining these complementary sources of information, each message is described from several perspectives rather than relying on individual words alone. The resulting representation contains 450 numerical features and serves as the input to the classification models described in the following sections.

Stage	Description	Output
Raw message	Original SMS message	Text
Text preprocessing	Message normalization described in Section 4.4	Cleaned text
Semantic representation	SBERT embeddings described in Section 5.2	384 features
Lexical representation	TF-IDF features described in Section 5.1	50 features
Binary and engineered features	Features presented in Chapter 5	16 features
Final representation	Combination of all feature groups	450 features



6.4 Used Models

The selection of classification models was guided by previous research on phishing and SMS smishing detection. A review of the literature revealed several machine learning approaches that consistently achieved strong performance across different datasets and feature representations. Based on these findings, five models were selected for evaluation in this study.

- **Logistic Regression** was included as a simple and interpretable baseline model that has been widely used in text classification problems.
- **Support Vector Machines (SVM)** have frequently demonstrated strong performance in phishing detection tasks and were therefore included in the comparison.
- **Random Forest** was selected as a representative tree-based ensemble method capable of capturing nonlinear relationships between features.

- **XGBoost** was chosen because gradient boosting methods have achieved state-of-the-art results in many studies involving structured feature sets.
- **Multi Layer Perceptron (MLP)** neural network was evaluated in order to examine whether a neural approach could capture additional relationships within the feature space.

Since each model has different strengths, a **weighted soft voting ensemble** was constructed to combine their predictions. Rather than relying on a single classifier, the ensemble integrates the probability estimates produced by all five models to obtain a final prediction. The performance of the ensemble model is discussed in Section 6.7.

Model	Motivation
Logistic Regression	Widely used interpretable baseline model
Support Vector Machine (SVM)	Strong performance reported in previous phishing studies
Random Forest	Robust tree-based ensemble method
XGBoost	Frequently achieves state-of-the-art performance on structured features
Multi-Layer Perceptron (MLP)	Evaluates the ability of neural networks to capture complex interactions
Weighted Soft-Voting Ensemble	Combines the strengths of all five models

6.5 Model Training Process

After constructing the 450 dimensional feature vectors, the dataset was divided into separate training and test sets. A stratified split was used in order to preserve the original proportions of legitimate and smishing messages in both subsets. This approach ensures that the evaluation is representative of the overall dataset.

The training set was used to train the models and optimize their parameters, while the test set was reserved exclusively for the final evaluation. Keeping the test data separate prevents information from future samples from influencing the learning process and provides an unbiased estimate of model performance.

Since the dataset contains fewer smishing messages than legitimate messages, special attention was given to the class imbalance problem. During training, the models were configured to account for this imbalance in order to avoid favoring the majority class and to improve the detection of malicious messages.

To obtain reliable performance estimates, **five fold stratified cross validation** was employed during model development. Instead of relying on a single split of the training data, this approach evaluates the models across multiple training and validation partitions, resulting in more stable and robust estimates.

Following the augmentation process described in Chapter 4, the final dataset contained 8,141 messages. Approximately 80% of the samples were used for training, while the remaining 20% were reserved for testing.

Dataset subset	Number of messages	Percentage
Training set	6,512	80%
Test set	1,629	20%
Total dataset	8,141	100%

6.6 Hyperparameter Tuning

The performance of machine learning models depends not only on the choice of the classifier itself, but also on the values of its hyperparameters. Therefore, each model was optimized through a systematic hyperparameter search in order to identify the configuration that provides the best overall performance.

Since the dataset is moderately imbalanced, accuracy alone may provide an overly optimistic assessment. For this reason, the F1 score was used as the optimization criterion. The F1 score combines precision and recall into a single measure and therefore provides a more meaningful assessment of phishing detection performance. A detailed description of the evaluation metrics is provided in the following section.

To obtain reliable estimates, the hyperparameter search was combined with **five fold stratified cross validation**. This approach evaluates each candidate configuration across multiple training and validation splits and reduces the influence of any particular partition of the data.

Model	Hyperparameters	Description
Logistic Regression	C	Controls the amount of regularization and the trade off between model complexity and generalization.
Support Vector Machine (SVM)	C, kernel	C controls regularization, while kernel determines the type of decision boundary used by the model.
Random Forest	n_estimators, max_depth	n_estimators specifies the number of trees, and max_depth limits the depth of each tree.
XGBoost	n_estimators, max_depth, learning_rate	These parameters control the number of trees, their complexity, and the contribution of each boosting step.
Multi-Layer Perceptron (MLP)	hidden_layer_sizes, alpha	hidden_layer_sizes determines the network architecture, while alpha controls regularization to reduce overfitting.

The best-performing configuration for each model was selected automatically and used during the final evaluation presented in the following sections.

6.7 Weighted Soft-Voting Ensemble

After optimizing the individual classifiers, a weighted soft-voting ensemble was constructed to combine their predictions. Rather than relying on a single model, the ensemble aggregates the probability estimates produced by all five classifiers.

In a soft-voting scheme, each model estimates the probability that a message belongs to the smishing class. These probabilities are then combined into a single probability value, which is used to determine the final prediction.

Since the individual classifiers achieved slightly different levels of performance, their contributions to the ensemble were weighted according to their F1 scores. Consequently, models with stronger performance had a greater influence on the final decision, while weaker models contributed less. This weighting strategy allows the ensemble to exploit the complementary strengths of different learning algorithms and reduces the dependence on any single classifier.

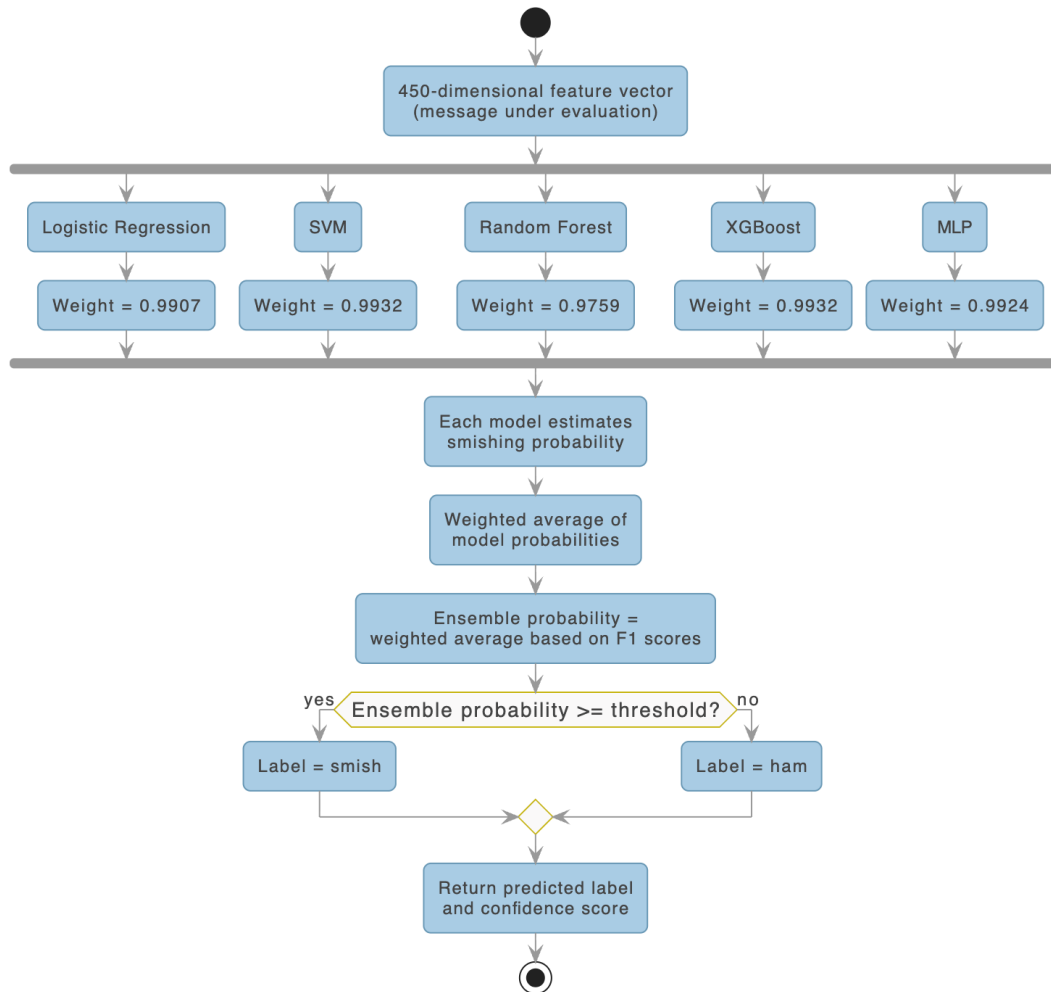
The final ensemble probability is computed as a weighted average of the probability estimates produced by the individual models:

$$P_{\text{ensemble}} = \frac{\sum_i w_i P_i}{\sum_i w_i}$$

where (P_i) denotes the probability estimated by model (i) , and (w_i) represents the weight assigned to that model according to its F1 score.

The performance of the ensemble was compared with that of the five individual classifiers. The model achieving the highest F1 score was selected as the final model used throughout the remainder of this study.

Weighted Soft-Voting Ensemble for SMS Smishing Detection



7. Model Evaluation

7.1 Evaluation Methodology

All performance results presented in this chapter were obtained using the held out test set, which consists of 20% of the augmented dataset. This subset was kept separate from the training process and was not used during hyperparameter tuning or cross validation.

Following the procedure described in Chapter 6, the dataset was divided into training and test sets while preserving the original class distribution. As a result, the training set contained 6,512 messages, whereas the remaining 1,629 messages were reserved exclusively for evaluation.

Each message in the test set was represented using the same 450-dimensional feature vector employed during training. Consequently, the reported results reflect the performance of the complete classification pipeline on previously unseen messages and provide an unbiased estimate of the model's expected behavior in real world use.

Class	Number of messages	Percentage
Ham	1,011	62.1%
Smish	618	37.9%
Total	1,629	100%

7.2 Evaluation Metrics

The performance of the classification models was assessed using several evaluation metrics derived from the confusion matrix. These metrics provide complementary information about different aspects of model performance and are particularly important when dealing with imbalanced datasets.

The confusion matrix summarizes the relationship between the actual class labels and the predictions produced by the model. It consists of four possible outcomes: true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).

	Predicted ham	Predicted smish
Actual ham	TN (True Negative)	FP (False Positive)
Actual smish	FN (False Negative)	TP (True Positive)

A true negative (TN) corresponds to a legitimate message correctly classified as legitimate, whereas a true positive (TP) corresponds to a smishing message correctly identified by the model. In contrast, false positives (FP) represent legitimate messages incorrectly flagged as smishing, while false negatives (FN) correspond to smishing messages that remain undetected.

The following metrics are computed from these four quantities and are used to evaluate the classification performance.

7.2.1 Accuracy

Accuracy measures the overall proportion of correctly classified messages. It is computed as the ratio between the number of correct predictions and the total number of messages. Although accuracy is easy to understand, it may provide an overly optimistic assessment when the classes are imbalanced. For this reason, additional metrics are also considered.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

7.2.2 Precision

Precision measures the proportion of messages predicted as smishing that are actually smishing messages. High precision indicates that the model generates relatively few false alarms and therefore minimizes the number of legitimate messages incorrectly flagged as malicious.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

7.2.3 Recall

Recall measures the proportion of actual smishing messages that are successfully detected by the model. In phishing detection, recall is particularly important because missed attacks may expose users to financial loss or credential theft.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

7.2.4 F1-Score

The F1-score combines precision and recall into a single metric. It provides a balanced measure of performance and favors models that perform well on both criteria simultaneously. For this reason, the F1-score was used as the optimization criterion during hyperparameter tuning.

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

7.2.5 Confusion Matrix

While the previous metrics summarize performance using a single numerical value, the confusion matrix provides a more detailed view of the model's behavior. In particular, it reveals the types of errors made by the classifier and distinguishes between false positives and false negatives. The confusion matrices presented later in this chapter provide additional insight into the strengths and weaknesses of each model.

7.3 Results of the Selected Model

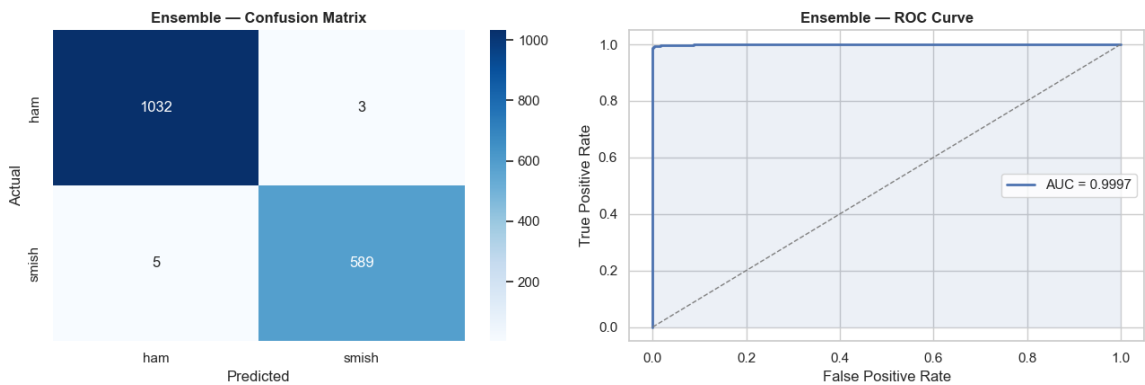
According to the methodology described in Chapter 6, the weighted soft voting ensemble was selected as the final model used in this study. Its performance was evaluated on the held out test set consisting of 1,629 previously unseen messages.

The evaluation results demonstrate that the ensemble achieved excellent classification performance across all metrics. In particular, the model obtained high precision and recall, resulting in an F1-score of 0.9933. In addition, the ROC-AUC value of 0.9997 indicates an excellent ability to distinguish between legitimate and smishing messages over a wide range of decision thresholds.

Metric	Value
Accuracy	0.9951
Precision	0.9949
Recall	0.9916
F1-score	0.9933
ROC-AUC	0.9997

The confusion matrix shows that the model correctly classified 1,032 legitimate messages and 589 smishing messages. Only three legitimate messages were incorrectly flagged as smishing, while five smishing messages were missed by the classifier.

Overall, the ensemble made only a small number of errors among the 1,629 test messages, demonstrating the effectiveness of combining semantic embeddings, lexical features, binary indicators, and handcrafted features.



7.4 Comparison Between Models

In addition to evaluating the selected ensemble model, the performance of all individual classifiers was also compared. This comparison provides insight into the strengths and weaknesses of the different learning approaches and demonstrates the contribution of the ensemble strategy.

Overall, all models achieved very high scores, indicating that the proposed feature representation provides strong discriminatory information for smishing detection.

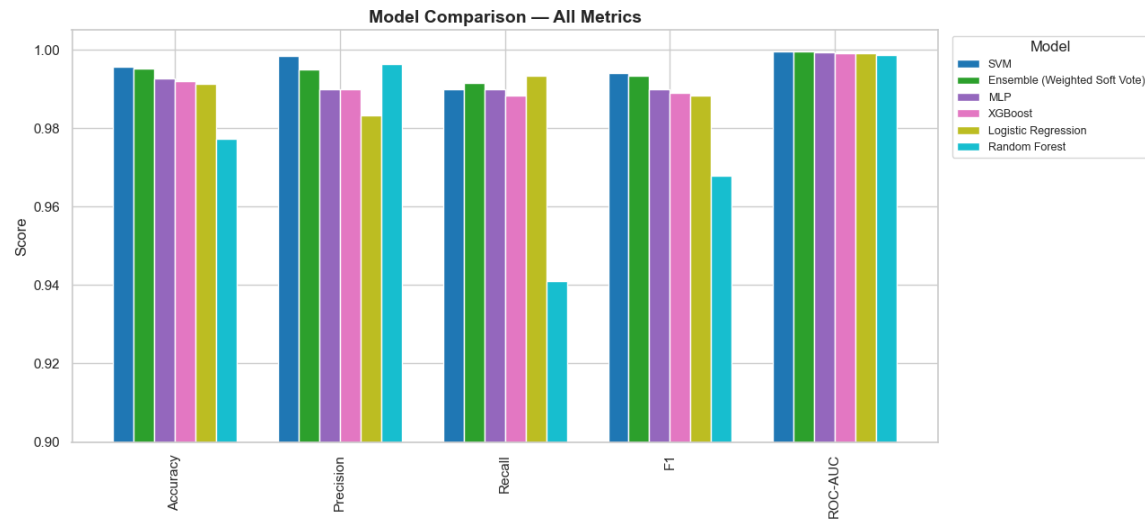
Among the individual classifiers, the Support Vector Machine (SVM) achieved the highest F1 score of 0.9941 and the highest overall accuracy. Logistic Regression obtained the highest recall, successfully identifying the largest proportion of smishing messages, although at the expense of a slightly lower precision. In contrast, Random Forest achieved the highest precision but exhibited noticeably lower recall, indicating that it missed a larger number of malicious messages.

The weighted soft voting ensemble achieved highly balanced performance across all evaluation metrics. Although its F1 score was slightly lower than that of the SVM model, it provided a better trade off between precision and recall and benefited from the complementary strengths of the individual classifiers.

Overall, the results show that no single model dominates all evaluation metrics. Different models emphasize different aspects of performance, and combining them through the ensemble approach leads to robust and stable predictions.

The following figure presents a visual comparison of the performance metrics achieved by the different models, while the following table provides the detailed numerical results.

	Model	Accuracy	Precision	Recall	F1	ROC-AUC	FPR	FNR
0	SVM	0.9957	0.9983	0.9899	0.9941	0.9997	0.0010	0.0101
1	Ensemble (Weighted Soft Vote)	0.9951	0.9949	0.9916	0.9933	0.9997	0.0029	0.0084
2	MLP	0.9926	0.9899	0.9899	0.9899	0.9994	0.0058	0.0101
3	XGBoost	0.9920	0.9899	0.9882	0.9890	0.9991	0.0058	0.0118
4	Logistic Regression	0.9914	0.9833	0.9933	0.9883	0.9991	0.0097	0.0067
5	Random Forest	0.9773	0.9964	0.9411	0.9680	0.9987	0.0019	0.0589



7.5 Results Analysis

The comparison between the six evaluated models shows that all classifiers achieved very strong performance. In particular, every model obtained an F1 score above 0.96, indicating that the 450-dimensional feature representation described in Chapter 5 provides highly informative and consistent signals for distinguishing between legitimate and smishing messages.

Nevertheless, several differences can be observed. Random Forest exhibited the lowest recall among the evaluated models. Although it achieved very high precision, it missed a larger number of smishing messages than the other classifiers. This behavior suggests a more conservative prediction strategy, where the model tends to avoid false alarms at the expense of missing some malicious messages.

At the other extreme, the Support Vector Machine achieved the highest F1 score and overall accuracy. However, the performance difference between the SVM and the weighted soft voting ensemble was very small. This result indicates that the proposed feature representation is sufficiently strong for several learning algorithms to achieve similar levels of performance.

Although the ensemble did not significantly outperform the best individual classifier, it provides a more robust and stable solution by combining the complementary strengths of multiple models. Consequently, the ensemble reduces the dependence on the behavior and limitations of any single classifier.

7.6 Conclusions from Model Evaluation

The experimental results demonstrate that the proposed weighted soft voting ensemble achieves excellent performance on previously unseen messages. In particular, the model obtained an F1 score above 0.99 and a ROC AUC value above 0.999, indicating a strong ability to distinguish between legitimate and smishing messages.

These results were obtained on a deliberately challenging dataset containing realistic legitimate financial messages that closely resemble phishing attempts. Therefore, the high performance achieved by the model suggests that the combination of semantic embeddings, lexical features, binary indicators, and handcrafted features provides an effective representation for SMS smishing detection.

Despite the excellent overall results, a small number of classification errors still remain. These errors are analyzed in greater detail in the next chapter in order to better understand the limitations of the proposed approach and identify possible directions for future improvements.

8. Error Analysis

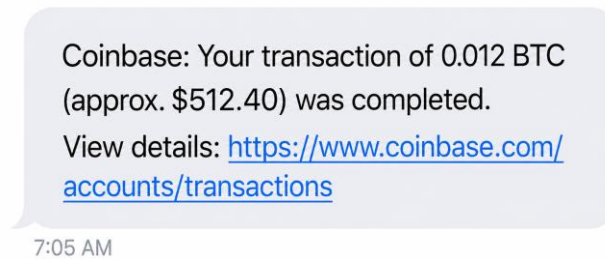
The evaluation results presented in Chapter 7 demonstrate that the proposed model achieves excellent overall performance. However, aggregate metrics alone do not explain the nature of the remaining classification errors. Therefore, a detailed error analysis was performed in order to better understand the limitations of the model and identify opportunities for future improvement.

Out of the 1,629 messages in the test set, only eight were misclassified, corresponding to an error rate of approximately 0.49%. These errors consisted of three false positives and five false negatives.

8.1 False Positives

False positives correspond to legitimate messages that were incorrectly classified as smishing. Although these messages were genuine, they contained characteristics commonly associated with phishing attacks, such as financial information, URLs, or references to official organizations.

For example, one misclassified message was a legitimate Coinbase transaction notification:



The message contains both a financial amount and a URL, two characteristics frequently observed in phishing campaigns. As a result, the model assigned it a relatively high probability of belonging to the smishing class.

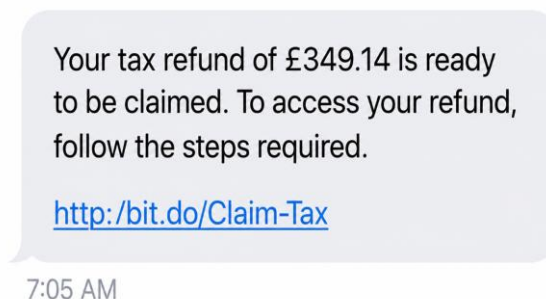
Another false positive was a legitimate notification referencing a court case update through an official government website. Although the message was genuine, its wording resembled the style commonly used in phishing messages that attempt to create urgency and encourage users to follow a link.

These examples illustrate the challenge of distinguishing between legitimate service notifications and sophisticated phishing attempts when both share similar linguistic and structural patterns.

8.2 False Negatives

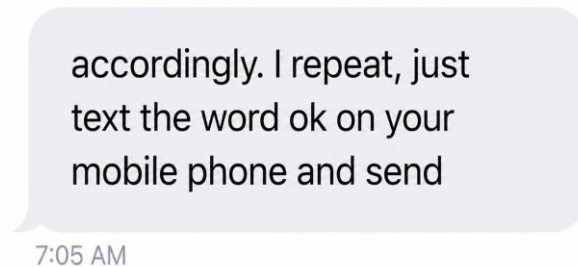
False negatives correspond to smishing messages that were incorrectly classified as legitimate. Since these messages represent attacks that remain undetected, they are considered the more critical type of error.

One representative example was a tax refund phishing message:



Although the message clearly attempts to lure the user into claiming a refund, its unusual formatting differs from many of the phishing examples seen during training, making it more difficult to identify.

Another false negative consisted of a short and incomplete message fragment:



This message provides very little context and appears to be incomplete. Consequently, even a human reader would find it difficult to determine its true intent.

These examples suggest that some false negatives are caused by unusual message structures or limitations in the quality of the underlying dataset rather than by systematic weaknesses of the classification model.

8.3 Analysis of Failure Cases

A closer examination of the misclassified messages revealed three recurring sources of error.

First, some legitimate financial notifications still resemble phishing messages closely enough to trigger a false alarm. Messages containing monetary amounts, account related terminology, and URLs remain particularly challenging because these features are common in both legitimate and malicious communications.

Second, certain phishing messages deviate from the patterns most frequently observed in the training data. Messages with unusual wording, formatting, or incomplete content may therefore be more difficult to classify correctly.

Finally, a small number of errors appear to be related to imperfections in the dataset itself. Truncated or ambiguous messages provide limited contextual information and may not be reliably classifiable, regardless of the learning algorithm used.

Overall, the remaining errors appear to be concentrated in highly ambiguous cases rather than resulting from a general failure of the proposed approach.

8.4 Lessons Learned from Error Analysis

The error analysis demonstrates that the proposed model is highly reliable, with fewer than one percent of the test messages being misclassified.

The results also indicate that the hard negative augmentation process described in Chapter 4 successfully reduced confusion between legitimate financial notifications and phishing messages. Nevertheless, a small number of challenging cases still remain, particularly when legitimate messages contain combinations of URLs, financial amounts, and action oriented language.

More broadly, the findings suggest that future improvements should focus primarily on expanding the dataset with additional difficult examples and improving data quality. Given the already strong performance of the classification models, these measures are likely to provide greater benefits than replacing the learning algorithm itself.

9. Backend Server

9.1 Backend Overview

The backend server is a Python based REST API server responsible for connecting the Android application with the trained machine learning model.

The server receives SMS message content from the Android application, passes it to the SMS analysis service, executes the trained classification model, and returns a prediction result indicating whether the message is legitimate or a smishing attempt.

The backend was implemented using FastAPI, which allows the system to expose REST API endpoints in a simple and efficient way. The backend also loads the trained machine learning model, the label encoder, the TF-IDF vectorizer, and the SBERT model during application startup, so they are available for incoming classification requests.

9.2 Technologies and Tools

Technology	Purpose
Python	Main programming language used
FastAPI	Python web framework used to implement the REST API server
Uvicorn	ASGI server used to run the FastAPI application
Sentence Transformers	Python library loading the SBERT model
Ngrok	DNS Secure Tunneling tool
Postman	API testing tool

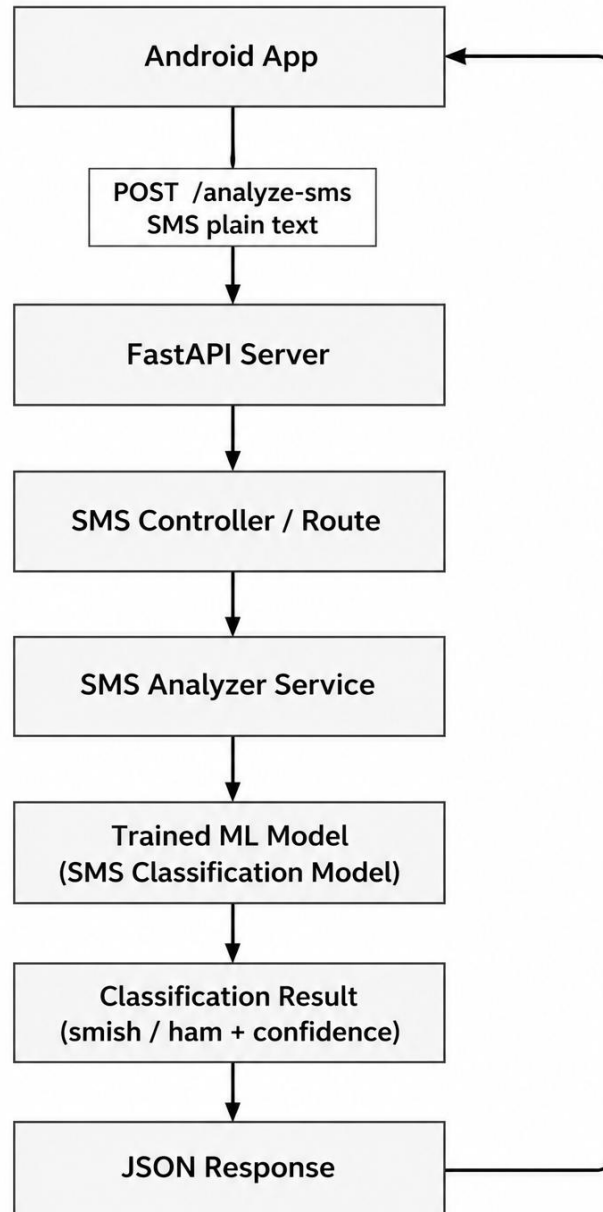
9.3 Backend Architecture

The backend follows a layered structure consisting of three main layers, Controller, Service, and Model.

The Android application communicates with the backend through REST API requests. Incoming SMS messages are received by the FastAPI server and routed to the appropriate controller. The controller forwards the request to the SMS Analyzer

Service, which executes the SMS classification process using the trained machine learning model.

Backend Architecture Diagram

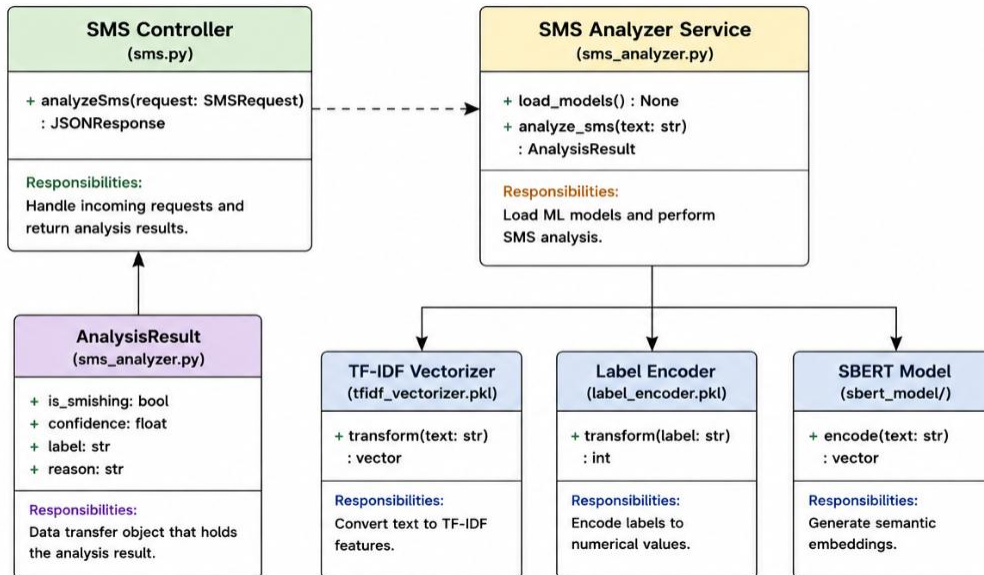


As shown in diagram, the Android application sends SMS content to the backend through the **POST /analyze-sms** endpoint. The request is handled by the **SMS Controller**, which passes the message to the **SMS Analyzer Service**. The service invokes the trained machine learning model and generates a classification result

consisting of a predicted label and a confidence score. Finally, the result is returned to the Android application as a structured JSON response.

9.4 Class Diagram UML

Backend UML Class Diagram



9.5 Main Components

9.5.1 Main Application

The main application file is responsible for creating and configuring the FastAPI application.

It defines the application title and version, configures CORS middleware, registers the SMS routes, and defines basic endpoints such as the root endpoint and the health check endpoint.

In addition, the backend uses a lifespan function that loads the machine learning models when the server starts. This ensures that the model, encoder, TF-IDF vectorizer, and SBERT model are loaded once and can then be reused for incoming SMS classification requests.

9.5.2 SMS Controller

Act as the controller layer of the backend.

This layer is responsible for receiving requests from the Android application and forwarding the SMS message content to the SMS analysis service. After the analysis is completed, the controller returns the prediction result to the client application.

This separation allows the API layer to remain focused on request handling, while the classification logic is handled separately by the service layer.

9.5.3 SMS Analyzer Service

The SMS Analyzer Service is responsible for executing the SMS classification process.

When the backend starts, this service loads the required model files, including the trained smishing detection model, the label encoder, the TF-IDF vectorizer, and the SBERT model.

When a new SMS message is received, the service calls the prediction function and passes the message together with the loaded model components. The prediction function returns a label and a confidence score.

The service then converts the prediction into a structured analysis result that includes the predicted label and a prediction confidence score

9.6 API Endpoints

The backend exposes several REST API endpoints used for communication and testing.

Method	Endpoint	Purpose
GET	/	Returns a basic message indicating that the SmishGuard API is running
GET	/health	Returns the server status and confirms that the backend is available
POST	/analyze-sms	Receives SMS content and returns a classification result

10. Android Application

10.1 Application Overview

The proposed solution includes an Android application that continuously runs in the background and monitors incoming SMS messages, subject to the permissions granted by the user.

When a new SMS message is received, the application sends the message content to the backend server for analysis. If the machine learning model classifies the message as a smishing attempt, the application immediately notifies the user through a warning notification.

The application also provides a user interface that allows users to view the application's protection status and SMS scanning statistics.

10.2 Technologies and Tools

Technology	Purpose
Kotlin	Programming language used for developing the Android application
Retrofit	HTTP client library used for communication with the backend REST API

10.3 Application Architecture

The Android application follows a layered architecture consisting of three main layers, UI Layer, Business Layer, and Data Layer.

This separation improves maintainability, readability, and scalability by assigning a clear responsibility to each layer. The UI Layer is responsible for presenting information to the user, the Business Layer contains the application's logic and workflow, and the Data Layer manages communication with external and local data sources.

10.4 Main Components

10.4.1 UI Layer

The UI Layer is responsible for interacting with the user and presenting information.

Main components:

1. MainActivity – The main screen of the application, displaying protection status and SMS scanning statistics.
2. PermissionActivity – Responsible for requesting the required SMS and notification permissions from the user.
3. HomeViewModel – Manages the UI state and provides data to the application's screens.

10.4.2 Business Layer

The Business Layer contains the application's business logic and serves as a bridge between the UI Layer and the Data Layer.

Main components:

1. SmsReceiver – Listens for incoming SMS messages using Android's BroadcastReceiver mechanism.
2. SmsAnalysisWorker – Executes SMS analysis tasks in the background using WorkManager.
3. SmsAnalysisRepository – Manages the SMS analysis process, sends messages to the backend server, and updates application statistics.
4. NotificationHelper – Creates and displays warning notifications when suspicious SMS messages are detected.

10.4.3 Data Layer

The Data Layer is responsible for managing application data and communication with both remote and local data sources.

Remote Data Source

1. ApiService – Defines the communication interface with the backend server.
2. RealApiService – Performs actual API requests to the backend classification service.
3. MockApiService – Provides simulated responses for testing purposes.
4. AnalysisRequest and AnalysisResponse – Data transfer objects used for communication with the backend server.

Local Data Source

1. StatsStore – Stores application statistics locally on the device, including the total number of scanned messages and the number of detected smishing messages.

10.5 Permissions Management

The Android application requires two permissions in order to operate correctly.

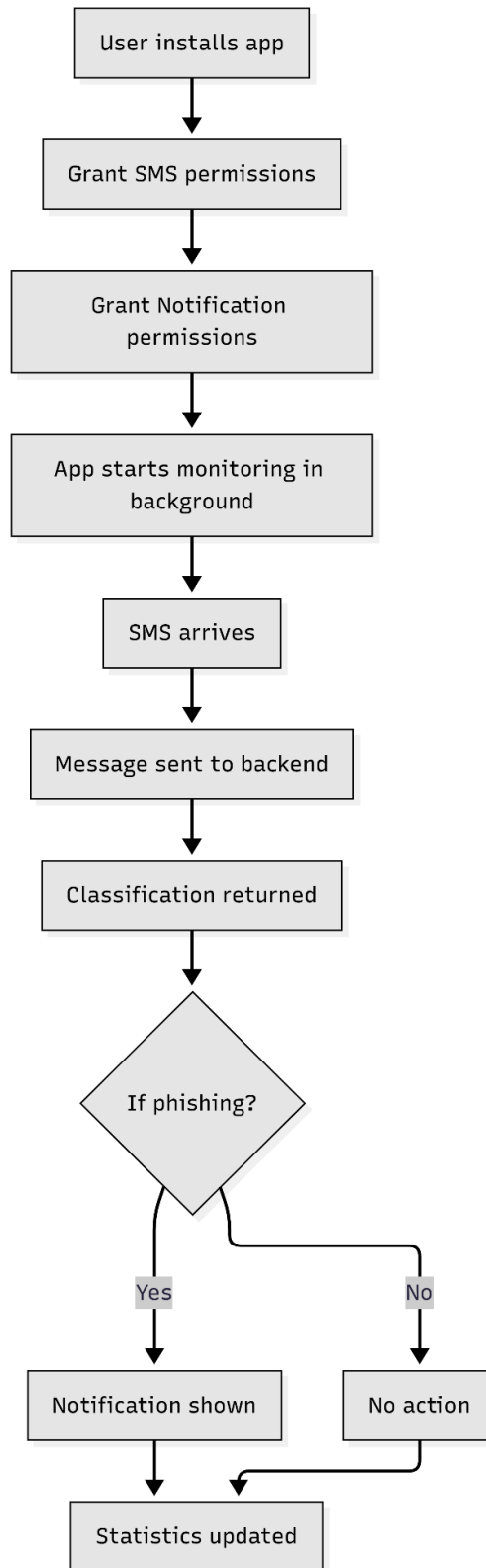
1. SMS Permission

This permission allows the application to receive and analyze incoming SMS messages. Without this permission, the application cannot monitor SMS activity or perform smishing detection.

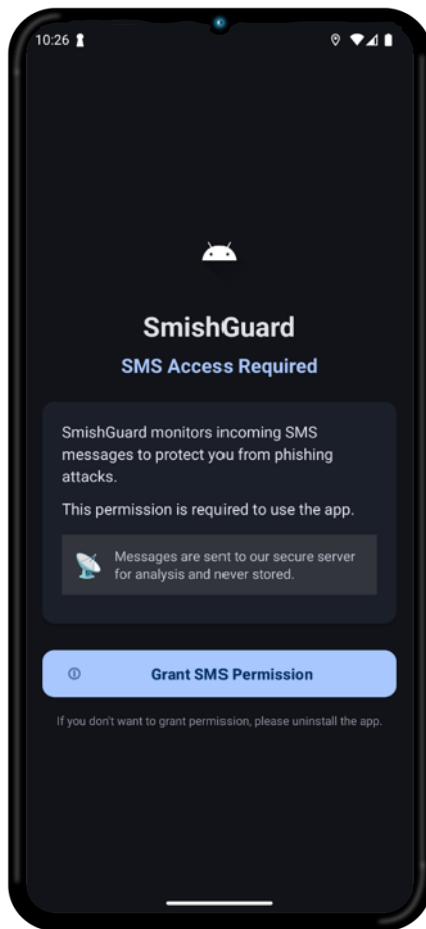
2. Notification Permission

This permission allows the application to display warning notifications when suspicious SMS messages are detected. Without this permission, the application can still analyze messages, but it cannot actively alert the user about potential threats.

10.6 User Interaction Flow

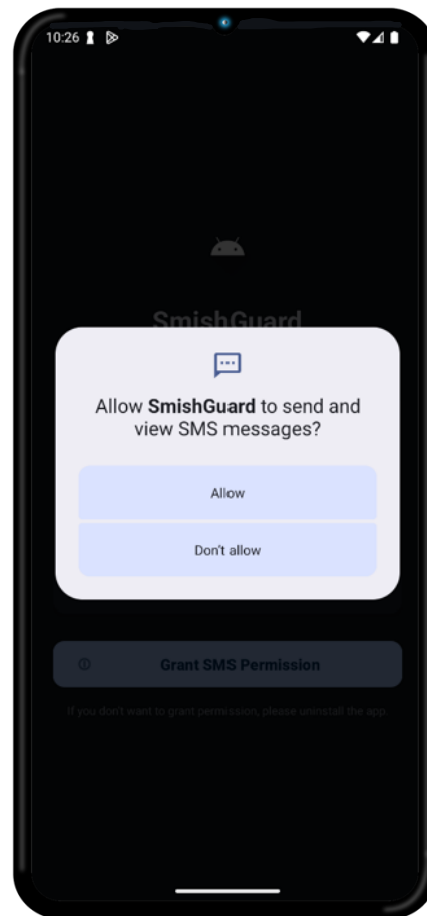


10.7 User Interface



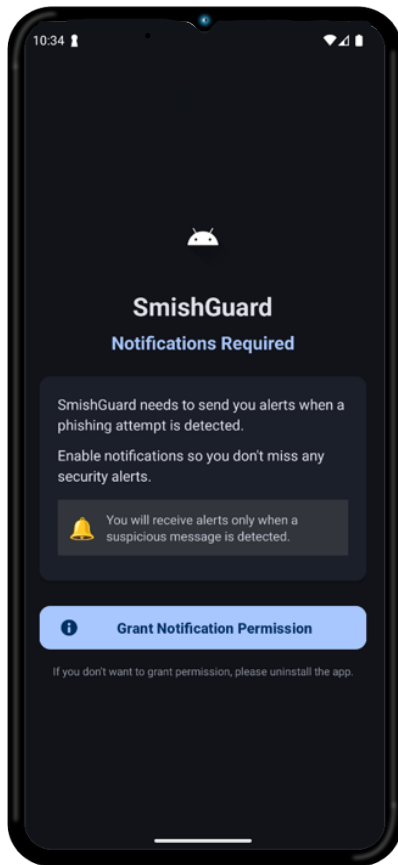
This is the first screen the user encounter to when he first installed or opened the app.

This screen requests SMS permissions from the user and explains why they are required.



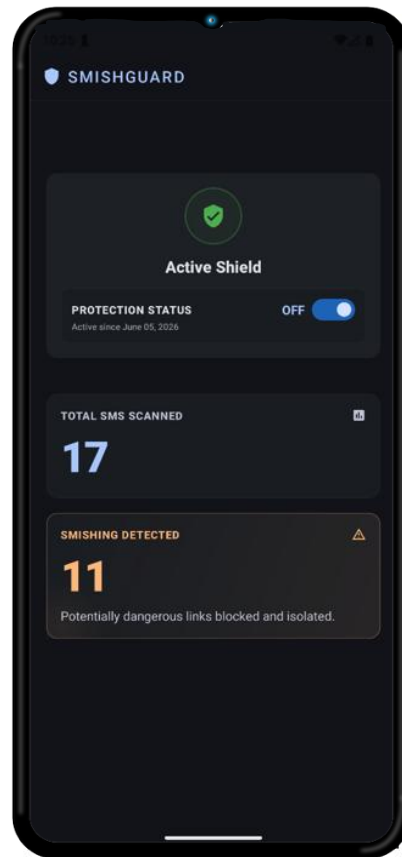
This screen appears after user press the Grant SMS permissions button, same will be for the notification's permissions.

This is an android screen and not related to our app.



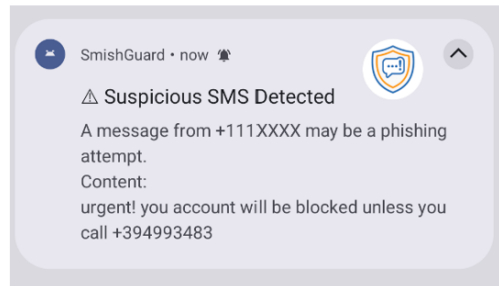
This screen too appears only if user first installed or opened app, or if he manually changed the permissions from settings.

This screen asks for permissions to send the user notifications and explaining the reason.



This is the main screen the user sees when he enters the app after first registration. This is some device scanning statistics.

The user can see how much SMS were scanned since he downloaded the app and how much were detected as potential smish.



This screen is the most important. This is the notification the user gets when the application suspects that an SMS might be smish.

The notification includes the phone number the user got the SMS from and the SMS content, so it will be very clear which SMS is the suspected one.

11. Development Challenges

During development we encountered some challenges that made this project harder, and we were required to understand how to overcome them.

11.1 Model and Dataset Challenges

As we mentioned before, our dataset was a generic smishing dataset taken from Kaggle, containing 5000~ rows, divided to smishing and ham.

While testing the model we realized our model is overfitting cause the validation results were very high, but when we used our own SMS examples, translated to English we realized our data was not diverse enough.

For example, all the SMS in this dataset were really simple. All hams SMS didn't contain any URLs, emails or phone, which immediately eliminates the possibility of them being smishing messages.

Without any link, phone number, email address, or any other way to redirect the user, it is much harder for a message to be used for a smishing attack.

Once we started testing our own ham messages, which contained some links, the model got confused and started detecting smish on all of them.

This is when we realized our data was problematic and started doing a lot of manipulation on it.

We generated using AI more messages, both ham and smish, that will look more like our own messages. We made sure to include ham messages with links.

We added more features, which we described in section 5, to try to overcome the problematic data we had.

After a lot of work, we accomplished a more diverse dataset, and better test results.

11.2 Android Development Challenges

The main challenge while developing on android is that we don't own an android phone with ESIM technology or physical sim.

This required us to work and test only on the android simulator, since it has a way to simulate SMS which doesn't exist in real devices.

The main problem is that the simulator has some known issues and get stuck a lot of times which made the development a bit harder and slower

12. Future Development

We have some ideas for future development that can make our project even better. I will describe them below

12.1 Support multiple SMS languages

This will make our project target a wider audience. Currently, our model is only trained on English but in the future, we can use an LLM to support multiple languages.

12.2 Self-Improving Classification Model

Our model dataset is currently static. We expanded it and tuned it to our needs, but we are not adding to it and retraining the model automatically. Our idea is to ask users to be active and for every notification arrival, clicking on a confirmation button that approves that this SMS is indeed fishing, and we will add it to the dataset with the fishing label. We will still have to add to our model ham messages, but at least we are doing half of the work automatically, and then we can occasionally train it and create a new model pickle file that our server will read.

13. Conclusion

The purpose of this project was to design and develop an intelligent system for real-time detection of SMS phishing attacks. The proposed solution combined machine learning techniques, a Python backend server, and an Android mobile application capable of monitoring incoming SMS messages and providing immediate warnings when suspicious messages were detected.

Throughout the project, detailed research was conducted on existing phishing detection approaches, machine learning techniques, and currently available market solutions. The research demonstrated that while several phishing detection systems already exist, many of them rely heavily on manual user interaction and do not provide fully automated real-time protection. This finding motivated the development of a passive detection system that operates automatically in the background.

A significant part of the project focused on dataset construction and feature engineering. During development, it became clear that publicly available datasets alone were insufficient for representing real world smishing attacks. As a result, additional real world examples and AI generated messages were incorporated into the dataset, improving its realism and helping the models generalize better to practical scenarios.

The final solution successfully integrates all planned components. The machine learning model analyzes incoming SMS messages, the backend server provides prediction services through a REST API, and the Android application delivers real-time notifications to users. Together, these components create a complete end-to-end smishing detection system.

The project also highlighted the importance of carefully handling false positives. By introducing realistic legitimate financial and service-related messages into the dataset, the system significantly reduced incorrect classifications of legitimate SMS messages while maintaining strong phishing detection capabilities.

Overall, the project achieved its primary objectives and demonstrated that machine learning can be effectively integrated into mobile security applications to provide automated protection against SMS phishing attacks.

13.1 Planned vs. Actual Implementation

At the beginning of the project, the planned solution consisted of three main components, a machine learning classification model, a backend server for hosting the model, and an Android application capable of monitoring SMS messages and notifying users about potential threats.

The final implementation successfully delivered all core components originally planned. The machine learning model was developed and trained using a combination of textual, semantic, and handcrafted features. A Python FastAPI backend was implemented to host the model and expose classification services through REST endpoints. In addition, an Android application was developed to receive incoming SMS messages, communicate with the backend server, and display warning notifications when suspicious messages were detected.

Several improvements were introduced during development that were not fully anticipated in the initial planning stage. The most significant enhancement was the expansion and refinement of the dataset through the addition of real world examples and AI generated messages. Furthermore, special attention was given to reducing false positives by creating hard-negative legitimate messages that closely resemble phishing attempts.

Some planned future capabilities were not implemented within the project timeframe. These include support for multiple SMS languages and a self improving model that continuously learns from new user feedback. These features remain opportunities for future development.

Despite these limitations, the final system fulfills all major functional requirements defined at the beginning of the project and provides a working solution for real time smishing detection on Android devices.

14. Bibliography

- 1] S. Mishra and D. Soni, "Smishing Detector: A Security Model to Detect Smishing Through SMS Content Analysis and URL Behavior Analysis," *Future Generation Computer Systems*, vol. 108, pp. 803–815, 2020.
- 2] Shaikh, M. Shaikh, and S. R., "Smishing Detection: Combating SMS Phishing Attacks by Utilizing Machine-Learning Algorithms," *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, vol. 14, no. 5, pp. 28–33, Apr. 2025.
- 3] T. Mahmud, M. A. H. Prince, M. H. Ali, M. S. Hossain, and K. Andersson, "Enhancing Cybersecurity: Hybrid Deep Learning Approaches to Smishing Attack Detection," *Systems*, vol. 12, no. 490, 2024.
- 4] A. K. Jain, K. Kaur, N. K. Gupta, and A. Khare, "Detecting Smishing Messages Using BERT and Advanced NLP Techniques," *SN Computer Science*, vol. 6, no. 109, 2025.
- 5] "Enhancing Mobile Cybersecurity: Smishing Detection Using Ensemble Learning and SMOTE," Preprint submitted to *Computers & Security*, May 2024. Available: SSRN.
- 6] F. Fatih,
"SMS Phishing Dataset,"
Kaggle.
Available: <https://www.kaggle.com/datasets/fadlifatih/sms-phishing-dataset>
[Accessed: Jan. 22, 2026]